Received: 3 March 2021 | Revised: 21 May 2021 | Accepted: 7 June 2021

DOI: 10.1002/cpe.6512

SPECIAL ISSUE PAPER

WILEY

Execution-Cache-Memory modeling and performance tuning of sparse matrix-vector multiplication and Lattice quantum chromodynamics on A64FX

Christie Alappat[1] | Nils Meyer[2] | Jan Laukemann[1] | Thomas Gruber[1] | Georg Hager[1] | Gerhard Wellein[1] | Tilo Wettig[2]

[1]Erlangen National High Performance Computing Center, Friedrich-Alexander-Universität Erlangen-Nürnberg, Erlangen, Germany

[2]Department of Physics, University of Regensburg, Regensburg, Germany

Correspondence
Christie Alappat, Erlangen National High Performance Computing Center, Friedrich-Alexander-Universität Erlangen-Nürnberg, Erlangen, Germany.
Email: christie.alappat@fau.de

Abstract

The A64FX CPU is arguably the most powerful Arm-based processor design to date. Although it is a traditional cache-based multicore processor, its peak performance and memory bandwidth rival accelerator devices. A good understanding of its performance features is of paramount importance for developers who wish to leverage its full potential. We present an architectural analysis of the A64FX used in the Fujitsu FX1000 supercomputer at a level of detail that allows for the construction of Execution-Cache-Memory performance models for steady-state loops. In the process we identify architectural peculiarities that point to viable generic optimization strategies. After validating the model using simple streaming loops we apply the insight gained to sparse matrix-vector multiplication (SpMV) and the domain wall (DW) kernel from quantum chromodynamics. For SpMV we show why the compressed row storage (CRS) matrix storage format is not a good practical choice on this architecture and how the SELL-C-σ format can achieve bandwidth saturation. For the DW kernel we provide a cache-reuse analysis and show how an appropriate choice of data layout for complex arrays can realize memory-bandwidth saturation in this case as well. A comparison with state-of-the-art high-end Intel Cascade Lake AP and Nvidia V100 systems puts the capabilities of the A64FX into perspective. We also explore the potential for power optimizations using the tuning knobs provided by the Fugaku system, achieving energy savings of about 31% for SpMV and 18% for DW.

KEYWORDS

A64FX, ECM model, Lattice quantum chromodynamics, sparse matrix-vector multiplication

1 | INTRODUCTION

The processor architectures used in HPC systems have been dominated for a long time by general-purpose commodity off-the-shelf processors (CPUs). Increasing clock speeds in the past and steadily increasing core counts in the last decade resulted in an attractive price-performance ratio at moderate power consumption. Traditional HPC-oriented architectures such as vector computers have almost been superseded. As power constraints and technology scaling limits became more pressing, a strong trend towards diversification in processor architectures for HPC started.

General-purpose graphics processing units (GPGPUs) provide new levels of price-performance and energy-per-flop efficiency and therefore have become very attractive for several application fields such as classical molecular dynamics, fluid dynamics, or linear solvers as well as artificial intelligence (AI).

Large initiatives have started to design custom HPC processors addressing the performance characteristics of a broad range of applications from computational science and engineering. They make use of new memory technologies or modular instruction sets and implement HPC-specific hardware concepts such as fast on-chip synchronization or specific on-chip accelerators. The Post-K and the European Processor Initiative projects are two such well-known endeavors. The former has already delivered the Fujitsu A64FX processor, which powers the fastest machine on the Top500 list as of November 2020, Fugaku.

The A64FX CPU is the second design (after Intel's Xeon Phi Knights Landing) that connects a classic cache-based multicore processor to high bandwidth memory (HBM). While the use of HBM is established on GPGPUs with their massively threaded programming and execution model, it is an interesting question if standard CPU-oriented programming models (e.g., OpenMP) in combination with the limited thread- and data-level parallelism of the CPU hardware can also exploit the potential of HBM. Several other features such as hardware barrier and sector cache have been implemented in the A64FX to address the needs of HPC as well as AI applications. At the same time, a strict power budget had to be kept, enforcing compromises in the design of cores, caches and the chip. Finally, the application performance of the A64FX critically depends on the quality of its rather new software ecosystem, in particular compilers and numerical libraries. This complex situation requires a careful analysis of existing well-optimized CPU codes, for example, to what extent they may exploit the benefits of the new design and how the new concepts implemented in the A64FX interfere with code-optimization techniques, parallelization strategies, and data layouts.

In this article, we use analytical performance modeling and the Execution-Cache-Memory (ECM) performance model to investigate and understand basic performance capabilities and new performance and power-saving features of the A64FX with a focus on streaming loops. In view of the CPU's high memory bandwidth (> 800 Gbyte/s) and moderate core count (48), the ECM model's capability to identify single-core performance contributions will be of central importance. We choose two case studies for in-depth application performance analysis representing important fields with moderate to low computational intensities: a sparse matrix-vector multiplication (SpMV) kernel and a Lattice quantum chromodynamics (QCD) domain wall (DW) kernel. Our analytical modeling approach allows us to pinpoint inefficiencies of the hardware design and the existing software ecosystem and provides recommendations on code optimization and data layouts. We further compare the performance characteristics of the A64FX with a high-end commodity server CPU system (Intel Cascade Lake AP) and a GPGPU (NVIDIA V100). The V100 uses a comparable HBM technology.

1.1 | Outline

The article is organized as follows: Section 2 describes the basic benchmarking methodology together with the compilers and libraries used. It further briefly summarizes the relevant performance characteristics of the standard CPU and GPGPU systems chosen for comparison. A detailed architectural analysis of the A64FX-based Fujitsu FX1000 used in the Fugaku system is provided in Section 3. Strong focus is put on the in-core analysis, including a discussion of the capabilities of Arm's Scalable Vector Extension (SVE) and the out-of-order (OoO) back end. Furthermore we discuss an additional feature set of the Fugaku system: the zero fill instruction which prevents write-allocate transfers, the hardware barrier, and the sector cache. In Section 4, we establish the ECM machine model for the A64FX and validate it for a broad range of streaming kernels. An analysis of SpMV performance on the A64FX is presented in Section 5. Starting with a standard CPU-friendly SpMV data format we identify shortcomings on the single-core level through the ECM model. We investigate the use of a vector-friendly data layout and of sector cache to fully exploit the available bandwidth. In Section 6, we address the large application field of Lattice QCD focusing on the DW kernel. The ECM model again guides the investigation of potential performance gains through code-optimization strategies and appropriate choices of data layout. The impact of A64FX's power-saving mechanisms and performance comparisons with standard CPU and GPGPU are presented in Sections 5 and 6 for both case studies. In Section 7, we summarize our findings and put our work in the context of existing literature.

1.2 | Extended version of workshop short paper

The work presented here is an extended version of a short paper published at the PMBS 2020 workshop.[1] The short paper investigated the basics of the ECM model and briefly demonstrated the benefit of a vector-friendly data layout for the A64FX processor used in the QPACE 4 (Fujitsu FX700) system. Both topics have now been investigated on the FX1000 system used in Fugaku. More importantly, we have substantially increased the scope of both topics, for example, by improving the ECM model considering the impact of page sizes and by presenting a detailed ECM model and performance-tuning strategies for SpMV. Topics presented here but not covered in Reference 1 include the case study of the Lattice QCD kernel,

the investigation of power-saving mechanisms and specific hardware features of the A64FX and the comparison with state-of-the-art CPUs and GPGPUs.

2 | TESTBED AND EXPERIMENTAL METHODOLOGY

The majority of this work was done on the Fugaku supercomputer. The system is running the Red Hat Enterprise Linux 8.3 operating system. The CPU supports two core clock frequencies, 2.0 and 2.2 GHz. The clock frequency was fixed to 2.2 GHz unless specified otherwise. The key specifications can be found in Table 1 and will be discussed in more detail in Section 3. All code was compiled with the GNU gcc (GCC 10.2.0) and Fujitsu tcsds 1.2.30 (FCC 4.4.0a) compilers. For GCC we used the `-march=armv8.2-a+sve` and `-Ofast` flags in combination with huge pages for compilation. For FCC there exist two modes of compilation, *Trad* and *Clang* mode. We used Trad mode with `-Kfast` and `-Khpctag` for all the runs, except for the DW QCD code where we used Clang mode with `-Ofast` due to the incompatibility of Trad mode with GCC attributes. Possible deviations from the default compiler flags stated above will be mentioned in the relevant sections.

All benchmarks were run in double precision so that a vector length (VL) of 512 bits corresponds to eight real or four complex elements. In general, SVE vector intrinsics (ACLE[2]) were employed to have better control over code generation. All arrays were aligned to OS page boundaries, for which best performance was observed in the experiments. In order to minimize statistical variations we repeated every benchmark loop for an overall runtime of at least 1 s. We do not show the statistical fluctuations if they were below 5%.

For benchmarking individual machine instructions we employed the ibench[3] framework and crosschecked our results with the A64FX Microarchitecture Manual.[4] The `LIKWID`[5] tool suite in version v5.1.0[6] was used, specifically `likwid-perfctr` for counting hardware events and `likwid-pin` to pin the threads to cores. The Power API framework[7] v2.0 installed on Fugaku was used for setting the power tuning knobs and measuring the energy consumption.

To put the results in relation to state-of-the-art hardware currently available, experiments were also run on an Intel Cascade Lake (CLX-AP) and an NVIDIA V100 GPU system. The CLX-AP experiments were conducted on a dual-socket Intel Xeon Platinum 9242 node with 96 cores and a STREAM TRIAD bandwidth of 464 Gbyte/s. For compilation the Intel compiler version 19.1.3 was used. The GPU experiments were performed on

TABLE 1 Key specifications of the A64FX CPU in the FX1000 system

Supported core frequency	2.0/2.2 GHz
Number of CMGs	4
Cores/threads per CMG	12/12
Instruction set	Armv8.2-A+SVE
Max. SVE vector length	512 bit
Peak flop rate	3379.2 Gflop/s
Cache line size	256 bytes
L1 cache capacity	48×64 KiB
L1 bandwidth per core ($b_{\text{Reg}\leftrightarrow\text{L1}}$)	128 B/cy LD ⊕ 64 B/cy ST
L2 cache capacity	4×8 MiB
L2 bandwidth per core ($b_{\text{L1}\leftrightarrow\text{L2}}$)	64 B/cy LD ⊕ 32 B/cy ST
Memory configuration	4×8 GiB HBM2
CMG TRIAD bandwidth	213 Gbyte/s
CMG read-only bandwidth	227 Gbyte/s
Full chip TRIAD bandwidth	841 Gbyte/s
Full chip read-only bandwidth	859 Gbyte/s
L1 translation lookaside buffer	16 entries
L2 translation lookaside buffer	1024 entries

Note: "⊕" represents an exclusive OR.
Abbreviations: CMG, core memory groups; SVE, Scalable Vector Extension.

an NVIDIA Tesla V100 connected via PCI-Express. The GPU kernels were compiled using CUDA v10.2 with GCC 8.1.0 as host compiler. The V100 GPU achieves a STREAM TRIAD bandwidth of 840 Gbyte/s, similar to that of A64FX.

3 | ARCHITECTURAL ANALYSIS

3.1 | In-core

To get a better understanding of the in-core behavior of the A64FX microarchitecture, a schematic block diagram of one core's front-end and back-end components is shown in Figure 1. Instructions are fetched from the L1 instruction cache with a bandwidth of 32 byte/cy. Each core's front end has an instruction buffer with 6×8 entries, which can feed six instructions, also called *macro-operations*, per cycle to the decoder. The decoder feeds up to four microoperations (μ-ops) per cycle to the different reservation stations (RSs), which then schedule the μ-ops to the corresponding execution pipelines. The reservation stations RSA0 and RSA1 are used for address generation and load units and have ten entries each, while reservation stations RSE0 and RSE1 are used for arithmetic execution and store units and have 20 entries each. The reservation station RSBR with its corresponding pipeline is only used for branching and has 19 entries. Excerpts of the execution units of the residual pipelines FL[A|B], PR, EX[A|B], EAG[A|B] are shown in boxes below the pipeline name. While RSA0 and RSA1 can schedule instructions to both EAG pipelines, all other reservation stations are limited to their corresponding pipelines, even if an execution unit with equivalent functionality exists in a pipeline outside their scope. Load (LD) and store (ST) instructions are fed to the fetch port and store port, respectively, which execute the requests in parallel to the operation flow.

The small capacity of the reservation stations in combination with high instruction latencies can result in inefficient OoO execution, which emphasizes the importance of a compiler that is capable of exploiting the in-core performance by intelligent code generation. While these hardware constraints cannot be overcome completely, their impact can be alleviated by techniques such as loop unrolling, consecutive addressing and interleaving of different instruction types. These techniques have been beneficial in our benchmarks, see Sections 4 and 5 for details. Neither the open-source GCC compiler (version 10.2.0) nor Fujitsu's proprietary FCC compiler (version 4.4.0a) currently generate code that fully overcomes the hardware constraints, which is why we employed SVE intrinsics for our benchmarks.

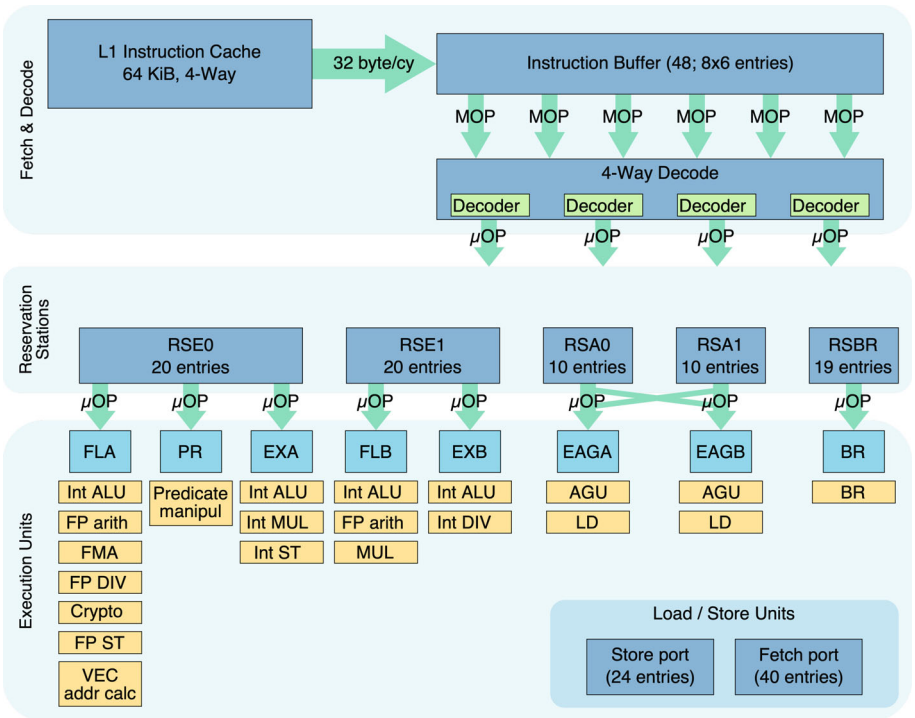


FIGURE 1 Overview of the in-order front end (*Fetch and Decode*) and out-of-order back end (*reservation stations* and *execution units*) components of an A64FX core based on Reference 4

TABLE 2 In-core instruction throughput and latency (if applicable) for selected instruction forms

Instruction	Reciprocal throughput (cy)	Latency (cy)
`ld1d` (standard)	0.5	11
`ld1d` (gather, simple stride)	2.0	≥ 11
`ld1d` (gather, complex stride)	4.0	≥ 11
simple gather + standard load	3.5	–
complex gather + standard load	5.5	–
`st1d` (standard)	1.0	–
`fadd`	0.5	9
`fmad`	0.5	9
`fmla`	0.5	9
`fmul`	0.5	9
`fcadd`	1.0	15
`fcmla`	2.0	16
`fadda` (512 bit)	18.5	72
`faddv` (512 bit)	11.5	49
`while{le|lo|ls|lt}`	1.0	1

To create an accurate in-core model of the A64FX microarchitecture, we analyze different *instruction forms*, that is, assembly instructions in combination with their operand types, based on the methodology introduced in References 8,9.* Table 2 shows a list of instruction forms relevant for this work. Standard SVE load (`ld1d`) instructions have a reciprocal throughput of 0.5 cycle (cy), while stores (`st1d`) have 1 cy. The throughput of gather instructions depends on the distribution of addresses: "simple" access patterns are stride 0 (no stride), 1 (consecutive load) and 2, while larger strides and irregular patterns are considered "complex." The former have lower reciprocal throughput and latency than the latter. However, when occurring in combination with a standard LD for loading the index array, we can observe an increase of reciprocal throughput by 1.5 cy instead of the expected 0.5 cy. This is caused by the dependency of the gather instruction on the preceding index load operation, which the OoO execution cannot hide completely. Note also the rather long latencies for arithmetic operations such as MUL, ADD, and FMA compared with other state-of-the-art architectures (e.g., on Intel Skylake or AMD Zen2 these are between 3 and 5 cy).

The SVE instruction set introduced a "`while`{cond}" instruction to set predicate registers according to the elements in vector registers in a length-agnostic way in order to eliminate remainder loops. Although it is used extensively for SVE code, a port-conflict analysis revealed that this instruction does not collide with floating-point instructions or data transfers.

3.2 | Chip topology and memory hierarchy

The chip is divided into four *core memory groups* (CMG) of twelve cores each. Every CMG is its own cache-coherent nonuniform memory access (ccNUMA) domain. The 64 KiB L1 cache is core-local, while 8 MiB of L2 are shared among the cores of a CMG. We refer to Table 1 for architectural details of the A64FX processor in the Fugaku system.

While parallel load/store from and to L1 cache is possible for general-purpose and NEON registers, different types of SVE data-transfer instructions in L1 cannot be executed in one cycle: Using SVE, one A64FX core can either load up to 2 × 64byte/cy or store 64 byte/cy from/to L1. The L2 cache can deliver 64 byte/cy to one L1 but tops out at 512 byte/cy per CMG. The L1-L2 write bandwidth is half the load bandwidth, that is, 32 byte/cy per core, and is capped at 256 byte/cy per CMG. Finally, the measured main-memory bandwidth per CMG is 227 Gbyte/s for read-only and 213 Gbyte/s for STREAM TRIAD. The memory bandwidth scales almost linearly with the CMGs and reaches a full-chip read-only bandwidth

*See github.com/RRZE-HPC/OSACA for a full set of measured instruction forms and the A64FX port model used in this work.

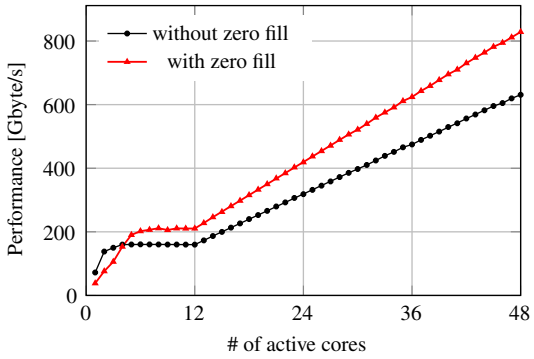


FIGURE 2 Reported STREAM TRIAD bandwidth with and without zero fill. The data-set size was 3 GB

of 859 Gbyte/s and STREAM TRIAD bandwidth of 841 Gbyte/s. These measured bandwidths will be used as baselines for the memory-transfer bandwidth in the ECM model.

3.3 | Special features of A64FX

The A64FX processor has some special hardware capabilities to improve the performance of some codes. In this section, we look into three features, that is, zero fill, hardware barrier, and sector cache. Currently only the FCC compiler supports these features on high-level code.

3.3.1 | Zero fill

A cache write miss causes a write-allocate transfer, that is, the cache line must be read before it can be modified. However, this increases the memory traffic, thus reducing the effective bandwidth available to the application. Most processors therefore have a mechanism to avoid this additional traffic. On the A64FX processor a similar mechanism exists and is called zero fill. The zero fill instruction `DC ZVA` directly writes a cache line filled with zeros to the L2 cache. Therefore, the processor can load the cache line from L2 through L1 avoiding the read operation from main memory. This effectively increases the measured application bandwidth.

For simple codes the FCC compiler is capable of automatically detecting the arrays which only have write operations and will use the `DC ZVA` instruction. In order to enable this the `-Kzfill` compiler flag has to be used. Figure 2 shows the increase in application bandwidth when using zero fill for the STREAM TRIAD (`a[i]=b[i]+s*c[i]`) benchmark. The benchmark reads two double-precision arrays and writes to one array. The bandwidth reported by the benchmark assumes 24 bytes of data traffic per iteration (byte/it), which in reality is obtained only if the write-allocate operation is avoided. Without zero fill there would be an additional read operation costing an extra 8 byte/it, and thereby we only observe 3/4th of the actual bandwidth. This difference can be seen in the figure.

3.3.2 | Hardware barrier

Another feature of the A64FX processor is the hardware barrier. The hardware barrier allows for fast synchronization of threads using dedicated system registers. With the FCC compiler the hardware barrier can be activated by setting the environment flag `FLIB_BARRIER` to `HARD`.

To determine the cost of the barrier we measured the difference in time between two variants of a computationally intensive kernel (calculating the exponential of a number), one with `omp barrier` and the other without. Figure 3 compares the cost (in cycles) of hardware barrier and different types of software barriers for varying number of threads. The cost of the default software barrier used by the GCC and FCC compilers is shown in blue and red, respectively. We see that FCC has a small advantage here. However, when we direct FCC's software barrier to use a spin waiting loop by setting the environment variable `OMP_WAIT_POLICY` to `active`, we see that the cost of the barrier drops further by 20%. The cost of GCC's software barrier did not change by setting this environment variable. We see that the hardware barrier performs best and requires only 550 cy within one CMG. Going beyond the CMG (12 cores) the cost increases to almost 2200 cy. This is because the hardware barrier is only implemented within a CMG, while between CMGs a software barrier is used. Note that the results shown here are the statistics from ten runs as the run-to-run fluctuations in this experiment were higher than 5% for some cases.

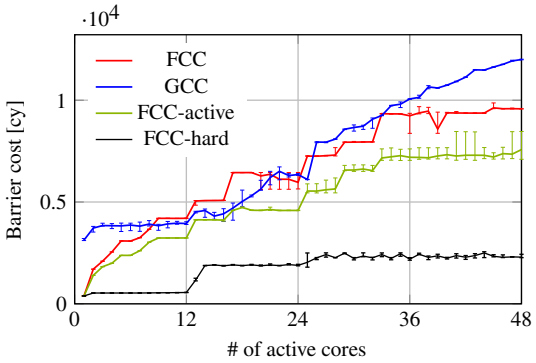


FIGURE 3 Comparison of the costs of different barrier implementations. GCC, FCC, and FCC-active refer to software barriers. The latter uses a spin waiting loop. FCC-hard uses the hardware barrier implemented on the A64FX

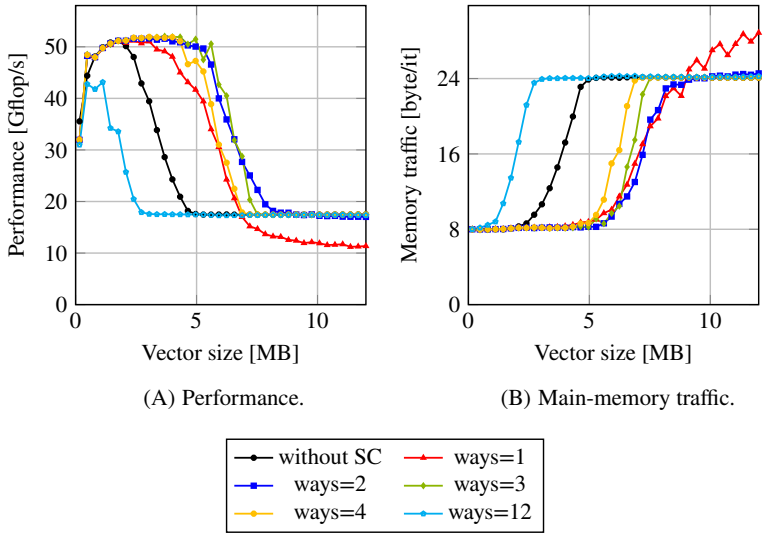
```
//loop over rows
for(int i = 0; i < N_r; ++i)
{
  //loop over columns
  for(int j = 0; j < N_c; ++j)
  {
    y[j] += A[i][j] * x[i];
  }
}
```

Listing 1: Code snippet of DAXPY-style DMVM

3.3.3 | Sector cache

Sector cache is a mechanism to partition a cache into different sectors of varying size. The application can then tag its data structures to be directed to one of the sectors. This allows for more control of the cache space allocated for each data structure in the code. For example, in case of a code with reuse on one array and streaming patterns on other arrays, one could direct the streaming arrays to a small sector of the cache to avoid polluting the cache space that could be used by the array having reuse. The sector size can be controlled with a granularity of cache ways. With the FCC compiler, pragma directives are used to activate the sector cache and to tag the arrays.

Figure 4 shows the impact of sector cache on a DAXPY-style dense matrix-vector multiplication (DMVM), for which the inner loop traverses a column of the matrix (see Listing 1). The DMVM kernel performs a multiplication of matrix A, stored in column-major format, with vector x and writes the result into vector y. The matrix array A does not have any reuse and therefore can be directed to one small sector of the cache. On the other hand, the vector array x is reused all the time, and the vector array y is reused if its size is small enough to fit in the cache. In the experiment we fix the number of rows to 192 and vary the number of columns, that is, x is fixed to length 192 and the length of y is variable. In Figure 4(A), we plot the performance as a function of the size of y. The different lines in the figure correspond to the different sizes of the cache sector allocated for matrix array A. The black line corresponds to the case without any use of sector cache. It can be seen that as the size of the vector y increases to about 2 MB the performance starts to drop as the vector y can no longer be kept in L2. The drop in performance can be correlated with an additional memory data traffic of 16 bytes (see Figure 4(B)) due to the read and write of vector y. However, if we restrict the space available to matrix A, the vector y has more cache space available, and therefore the kernel can sustain the high performance level until almost 5 MB. Restricting the cache available to matrix A to a very small size of 1 cache way is, however, not optimal since this leads to an early eviction of the prefetched elements of the matrix A. Obviously, the performance is also worse if we allocate almost all cache space (12 out of 14 allocatable ways) of L2 to the matrix A. Note that due to the high cost of the initial invocation of the sector cache (almost 500 ms) the first call to the DMVM kernel is not included in the performance results.



(A) Performance. (B) Main-memory traffic.

FIGURE 4 Performance and main-memory data traffic of dense matrix-vector multiplication for different vector sizes using sector cache

4 | CONSTRUCTION OF THE ECM MODEL

Given the information about in-core execution and data traffic across all data paths in the memory hierarchy gathered in Sections 3.1 and 3.2, a performance model for the A64FX can be constructed. The ECM model is an analytical performance model for streaming loop kernels with regular data-access patterns and equal amount of work per loop iteration, using first principles and machine-dependent constraints. As opposed to the roofline model (RLM), the ECM model can identify execution and data transfer bottlenecks for single-threaded programs and predict the scaling behavior of loops across the cores of a multicore chip. It also allows one to take into account overlapping or nonoverlapping data transfers within the cache hierarchy. The RLM always assumes full overlap of all time contributions.

4.1 | Time contributions

The ECM model of the A64FX contains four different time contributions:

1. $T_{\text{c\_OL}}$: execution time for in-core instructions that can overlap with data transfers. These are all instructions except loads. This also includes the cycles generated by the store instructions on the FLA and EXA pipelines.
2. $T_{\text{L1\_LD}}$: time for in-core load data traffic between registers and L1 cache.
3. $T_{\text{L1\_ST}}$: time for in-core store data traffic between registers and L1 cache.
4. Data transfer time between any other memory hierarchy levels: T_{L2} for data between L1 and L2, and T_{Mem} for data between L2 and main memory.

Combining these contributions, the single-core runtime prediction for the A64FX is defined as

$$T_{\text{ECM}} = \max\left(T_{\text{c\_OL}}, f(T_{\text{L1\_LD}}, T_{\text{L1\_ST}}, T_{\text{L2}}, T_{\text{Mem}})\right) , \tag{1}$$

where f is a combination of sum and max operators depending on the overlap hypothesis, which will be discussed in Section 4.2. For T_i with $i \in$ {c_OL, L1_LD, L1_ST} the time contributions are determined by a static analysis of the assembly code using the OSACA tool. For T_i with $i \in$ {L2, Mem} the time contributions are given by

$$T_i = V_i / b_i , \tag{2}$$

where V_i is the data volume transferred and b_i is the bandwidth between memory hierarchy level i and the next lower level.[†] The V_i include write-allocate transfers due to store misses where applicable. Latency effects are neglected.

[†] Lower means closer to the cores, for example, L1 is lower than L2.

4.2 | Overlap hypothesis

Depending on the architecture the data transfer contributions may or may not overlap, that is, the function f in Equation (1) has to be determined. In order to find out which of the time contributions for data transfers through the cache hierarchy overlap, measurements for a test kernel are compared with predictions based on different hypotheses, see Reference 10 for an in-depth description of this iterative process. If a hypothesis works for the test kernel, it is tested against a collection of other kernels with different characteristics to validate or invalidate the hypothesis.

Here, we use the STREAM TRIAD kernel, `a[i]=b[i]+s*c[i]`, to narrow down the possible overlap scenarios. This kernel has two LD, one ST, and one FMA instruction per SVE-vectorized iteration, which corresponds to eight high-level iterations. Figure 5(A) shows performance in cycles per VL for different code variants: "u = 1" denotes no unrolling (apart from SVE) and "u = 8" is eight-way unrolled on top of SVE. Some level of manual unrolling (typically eight-way) is usually required for best in-core performance. This is even more important in kernels where dependencies cannot be resolved easily by the OoO logic. Measurements of the STREAM TRIAD kernel with 64 KiB pages show performance degradation starting at 64 MiB due to TLB (translation lookaside buffer) misses after all 1024 entries of the L2 data TLB are used. The Fujitsu compiler suite uses 2 MiB large pages (also called *huge pages*) by default (`-Klargepage` option). For other compilers, the software environment on the Fugaku system provides the LIBMPG library and a custom linker script. TLB measurements of the STREAM TRIAD kernel with 2 MiB pages in Figure 5(A) show a rise in TLB misses when the working set size exceeds 2 GiB. Despite the occurrence of TLB misses, there is no observable drop in performance. For all further measurements in this work, a page size of 2 MiB is used.

Figure 6 compares four overlap scenarios (A)–(D) with measured cycles per VL (E). Note that there is a large number of possible overlap hypotheses, and we can only show a few here. The one leading to the best match with the STREAM TRIAD data (shown in Figure 6(D)) is the following:

- L1 is *partially* overlapping: Cycles in which STs are retired in the core can overlap with L1-L2 (or L2-L1) transfers, but cycles with LDs retiring cannot.
- L2 is *fully* overlapping: Cycles in which the memory interface reads and writes data from and to memory can entirely overlap with transfers between L2 and L1.

The overlap hypothesis (D) implies that the function f in Equation (1) has the form

$$f(T_{\mathrm{L1\_LD}}, T_{\mathrm{L1\_ST}}, T_{\mathrm{L2}}, T_{\mathrm{Mem}}) = \max\left(T_{\mathrm{L1\_LD}} + \max(T_{\mathrm{L1\_ST}}, T_{\mathrm{L2}}), T_{\mathrm{Mem}}\right) \ . \tag{3}$$

The overlap hypothesis (C) was used in previous work[1] and applies for standard 64 KiB pages.

In Figure 5(B), we show performance data and ECM model for a 2d five-point stencil, where SVE vectorization alone (without further unrolling) seems unable to resolve dependencies via OoO execution, leading to up to 2× slower code than the eight-way unrolled kernel despite the lack of loop-carried dependencies. Figure 5(C) shows performance data and ECM model for a sum reduction, `s+=a[i]`, which requires eight-way modulo variable expansion (MVE, see Section 5.2) on top of SVE in order to hide the large latency of the floating-point ADD instruction.

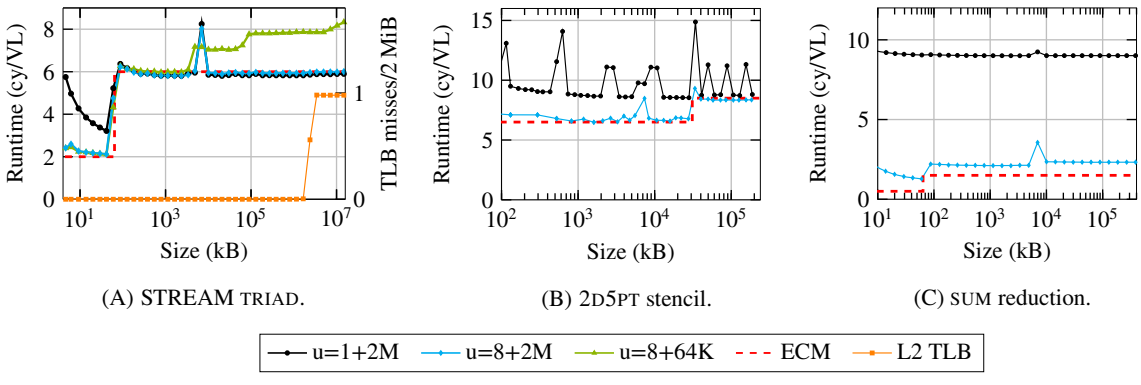


FIGURE 5 Runtime of SVE loop kernels versus problem size, comparing no unrolling (black) and eight-way unrolling (blue). Both versions are using 2 MiB huge pages. Arrays were aligned to 1024-byte boundaries. While huge pages were used by default, the extra green line denotes the usage of standard 64 KiB pages. The orange line in (A) shows the TLB misses on 2 MiB pages. For the 2D5PT stencil the outer and inner dimension was set at a ratio of 1:2. SVE, Scalable Vector Extension; TLB, translation lookaside buffer

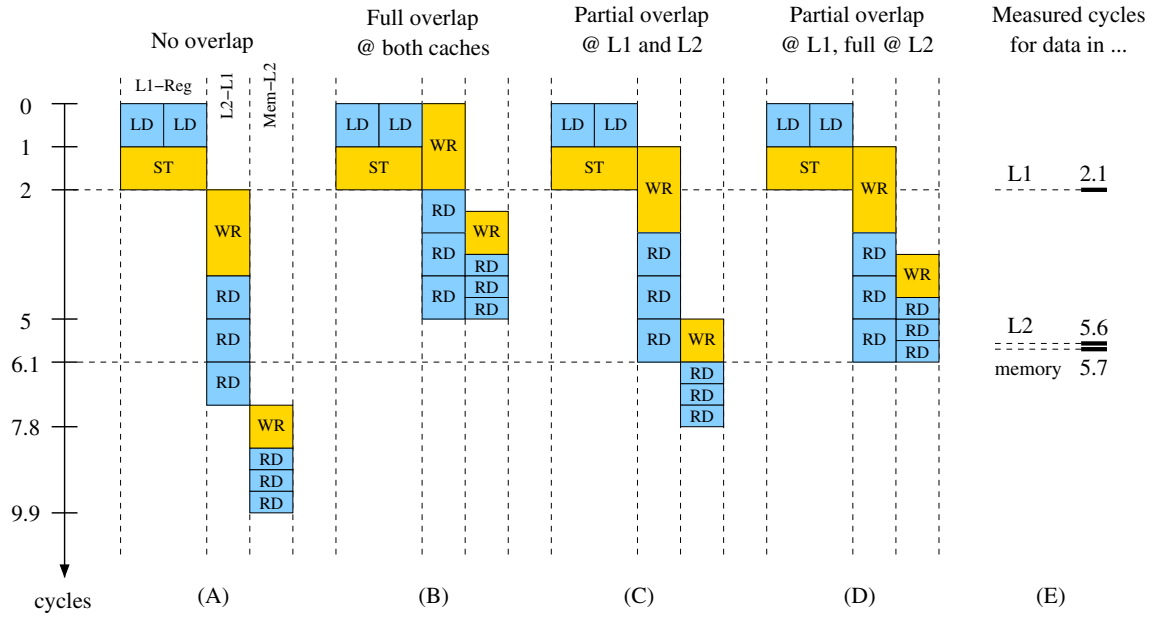


FIGURE 6 Comparing different overlap scenarios (A)–(D) for data transfers in the memory hierarchy with measured cycles per VL (E) on the STREAM TRIAD kernel. Note that (C) is the appropriate overlap hypothesis for standard (64 KiB) pages

4.3 | Multicore

For multiple cores within a CMG, the naive scaling hypothesis of the ECM model assumes perfect bandwidth scaling across all cores of the CMG until the main memory bandwidth bottleneck $b_{\text{Mem, CMG}}$ of the CMG is hit. If only a single core is active the memory bandwidth attained by a certain application is[‡]

$$B(1) = V_{\text{Mem}}/T_{\text{ECM}} \ . \tag{4}$$

Assuming linear scaling of the memory bandwidth with the number of active cores, the number N_s of cores required to attain the full bandwidth $b_{\text{Mem, CMG}}$ of one CMG is thus

$$N_s = \lceil b_{\text{Mem, CMG}}/B(1) \rceil \ . \tag{5}$$

Therefore, the ECM prediction can be extended to a function of a variable number n of cores,

$$T_{\text{ECM}}(n) = T_{\text{ECM}}(1)/\min(n, N_s) \ . \tag{6}$$

For $b_{\text{Mem, CMG}}$ we use the read-only bandwidth of 227 Gbyte/s if the application is dominated by loads and the STREAM TRIAD bandwidth of 213 Gbyte/s otherwise. If the application cannot saturate $b_{\text{Mem, CMG}}$, N_s will be larger than 12.

Going beyond a single CMG we again assume linear scaling since each CMG constitutes its own ccNUMA domain and each domain is connected to its own HBM stack. The full-chip memory bandwidths b_{Mem} for read-only and TRIAD are given in Table 1.

4.4 | Validation of the ECM model for streaming kernels

With the in-core and data-transfer models in place we can now test the ECM model against a variety of loop kernels. Table 3 shows a comparison of predictions and measurements. For each kernel, three numbers represent the cycles per VL with the dataset in L1, L2, and memory, respectively. INIT4 and LOAD4 are versions of INIT and LOAD with four independent data streams. In case of the 2d five-point stencil, three cases are shown: layer conditions (LC) satisfied at L1, broken at L1 and broken at L2.[§]

[‡]We use lowercase characters for architectural parameters and uppercase for observed or predicted quantities.

[§]See Reference 11 for a comprehensive coverage of LC in the context of the ECM model and Section 6.3.1 below.

TABLE 3 ECM model predictions and measurements in (cy/VL) for different streaming kernels on a single core

Kernel	Predictions	Measurements
COPY (`a[i]=b[i]`)	{1.5 ⌉ 4.5 ⌉ 4.6}	(1.6_{12} ⌉ 4.4_{13} ⌉ 4.6_{2})
DAXPY (`y[i]=a[i]*x+y[i]`)	{2.0 ⌉ 5.0 ⌉ 5.1}	(2.1_{8} ⌉ 4.7_{16} ⌉ 4.7_{12})
DOT (`sum+=a[i]*b[i]`)	{1.0 ⌉ 3.0 ⌉ 3.1}	(**1.7_{8}** ⌉ 3.2_{4} ⌉ 3.3_{3})
INIT (`a[i]=s`)	{1.0 ⌉ 3.0 ⌉ 3.1}	(1.0_{13} ⌉ 2.9_{13} ⌉ **4.0_{7}**)
INIT4 (`a[i]=s`)	{4.0 ⌉ 12.0 ⌉ 12.3}	(4.1_{2} ⌉ 10.6_{16} ⌉ 10.6_{16})
LOAD (`load(a[i])`)	{0.5 ⌉ 1.5 ⌉ 1.5}	(**0.7_{10}** ⌉ **2.3_{4}** ⌉ 1.5_{1})
LOAD4 (`load(a[i])`)	{2.0 ⌉ 6.0 ⌉ 6.1}	(**2.5_{4}** ⌉ 5.8_{16} ⌉ 5.9_{1})
TRIAD (`a[i]=b[i]+s*c[i]`)	{2.0 ⌉ 6.0 ⌉ 6.1}	(2.1_{8} ⌉ 5.6_{11} ⌉ 5.7_{1})
SUM (`sum+=a[i]`)	{0.5 ⌉ 1.5 ⌉ 1.5}	(**1.1_{11}** ⌉ **2.0_{15}** ⌉ **2.3_{9}**)
SCHÖNAUER (`a[i]=b[i]+c[i]*d[i]`)	{2.5 ⌉ 7.5 ⌉ 7.7}	(2.6_{14} ⌉ 7.0_{4} ⌉ 7.0_{4})
2D5PT - LC satisfied	{3.5 ⌉ 6.5 ⌉ 6.5}	(**5.8_{10}** ⌉ 6.5_{6} ⌉ 6.5_{9})
2D5PT - LC violated in L1	{3.5 ⌉ 8.5 ⌉ 8.7}	(**5.8_{10}** ⌉ 8.6_{7} ⌉ 8.4_{8})
2D5PT - LC violated	{3.5 ⌉ 8.5 ⌉ 8.7}	(**5.8_{10}** ⌉ 8.6_{7} ⌉ 8.4_{8})

Note: Bold indicates a deviation from the model of at least 15%. The optimal unrolling factor for each measurement is shown as a subscript.
Abbreviation: ECM, Execution-Cache-Memory; LC, layer conditions.

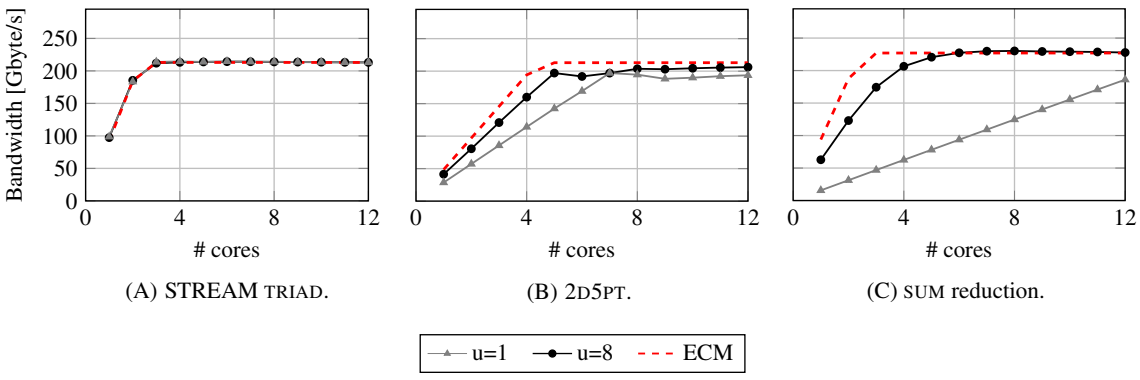


FIGURE 7 Strong scaling (constant working-set size) within one CMG for STREAM TRIAD, 2D5PT, and SUM kernels, comparing ECM model with measurements. Data without unrolling are shown for reference. Note that the read-only memory bandwidth was used as a limit for SUM. The working set size for TRIAD and SUM was set to 4 GB. For 2D5PT, the problem size was chosen as $10,000^2$ so that the layer condition is broken at L1 but fulfilled at L2. CMG, core memory groups; ECM, Execution-Cache-Memory

The results have been obtained by running each kernel with unrolling factors from 1 to 16 and taking the best result. Entries in boldface have a deviation from the model of 15% or more. The strongest deviations occur in L1: Even with massive MVE, SUM cannot achieve the architectural limit of 0.5 cy/VL. A similar deviation can be observed for the stencil kernels. We attribute this failure to insufficient OoO resources: A modified stencil code without intraiteration register dependencies achieves a performance within 10% of the prediction. Deviations from the model with L2 and memory working sets occur mainly with kernels that have a single data stream. In fact, we can observe that the ≥ 15% deviation for both LOAD and INIT in L2 and memory, respectively, decreases to 3% and 14% when using four streams.

We now move from single-core to multicore analysis within one CMG. Figure 7 shows a comparison of the ECM-model predictions and measurements for the STREAM TRIAD, 2D5PT, and SUM kernels. While STREAM TRIAD matches the prediction perfectly, for SUM it is evident that insufficient MVE (as shown in the "u = 1" data) is the root cause for nonsaturation of the memory bandwidth due to the long ADD latency. For the stencil kernel, saturation is possible even without unrolling, but more cores are needed.

5 | CASE STUDY: SPMV

SpMV is arguably one of the most relevant numerical kernels in computational science. With the help of the insights gained in the construction of the ECM performance model, we are now going to analyze and optimize the performance of SpMV kernels on the A64FX. We restrict ourselves to general matrices without the option of exploiting symmetries or dense substructures.

Due to its low computational intensity of at most 1/6 flop/byte[12] (assuming double precision and four-byte indexing), SpMV is typically expected to be memory bandwidth bound on all modern computer architectures if the matrix does not fit into cache. Hence, the OpenMP-parallel kernel (i) should exhibit the typical saturating scaling characteristics of a memory-bound code across the cores of a CMG, (ii) should be able to exhaust the available memory bandwidth, and (iii) should preferably show the maximum possible computational intensity as derived in Reference 13.

In most algorithms, a left-hand-side vector y is updated in the course of the SpMV operation: $y = y + Ax$. Due to the lack of store misses, zero fill instructions are thus unable to improve the performance here. However, since there is no cache reuse in the access to the matrix data but only in the right-hand-side vector, the sector-cache feature may be able to restrict the cache usage of the matrix, leaving more cache for the vector and thus helping to get close to the maximum intensity.

5.1 | Motivation

```
// parallel loop
for(int i = 0; i < N_{r}; ++i)
{
  for(int j = rp[i]; j < rp[i+1]; ++j)
  {
    y[i] += val[j] * x[ci[j]];
  }
}
```

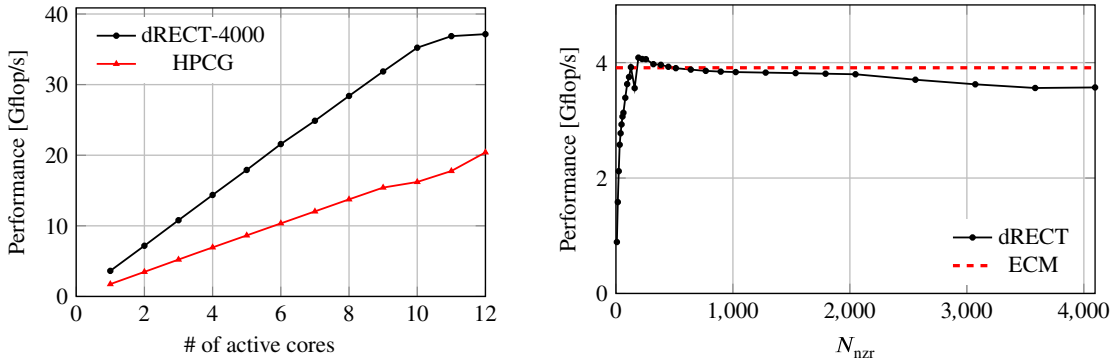
Listing 2: CRS SpMV kernel. The array rp[] holds the starting indices of the rows, while the array ci[] holds the column indices of the nonzeros

```
// parallel loop
for(int i = 0; i < N_{r}/C; ++i)
{
  for(int j = 0; j < cl[i]; ++j)
  {
    for(int k = 0; k < C; ++k)
    {
      y[i*C+k] += val[cs[i]+j*C+k] * x[col[cs[i]+j*C+k]];
    }
  }
}
```

Listing 3: SELL-C-σ SpMV kernel. The array cl[] holds the widths of the chunks, while the array cs[] holds their starting indices

In order to provide a baseline for experiments with realistic sparse matrices, we start with a "tall and skinny" dense rectangular (dRECT) matrix stored in the Compressed Row Storage (CRS) format, also called Compressed Sparse Row (CSR). CRS is the most popular sparse-matrix format, and it is usually well-suited for cache-based multicore CPUs. Listing 2 shows the corresponding high-level loop code. The dRECT matrix poses no challenges in terms of load balancing and right-hand-side access. The black line with symbols in Figure 8(A) show performance scaling on one CMG for a dRECT matrix with 4000 columns,¶using GCC with plain C code. It saturates at about 37 Gflop/s, which translates to a memory bandwidth of about 220 Gbyte/s assuming the maximum intensity of 1/6 flop/byte. Hence, we observe the expected pattern, although almost all cores are needed for saturation. In Figure 8(B), we show a scan of the single-core SpMV performance with the dRECT matrix with respect to the number of nonzeros per row. The sharp drop towards small N_{nzr} reflects the inefficiency of short inner loops, which we will elaborate on later.

¶For general sparse matrices, N_{nzr} is the average number of nonzeros per row, which is usually much smaller than the number of columns. In the special case of dRECT, all rows have the same number of nonzeros, which equals the number of columns.



(A) Strong scaling of SpMV with the dRECT matrix ($N_{nzr} = 4000$) and the HPCG matrix (problem size 128^3) using the CRS format across the cores of a CMG.

(B) Single-core performance of SpMV with the dRECT matrix versus N_{nzr}. The ECM model prediction is shown for reference. The working set size for the matrix was kept constant at 500 MiB.

FIGURE 8 Performance of SpMV with CRS format, compiled using GCC. CRS, compressed row storage; SpMV, sparse matrix-vector multiplication

Unfortunately, the dRECT case is not representative of most realistic sparse matrices, even for those with "benign" structures. The red line in Figure 8(A) shows performance scaling for the HPCG matrix (problem size 128^3, $N_{nzr} = 27$). In this case, the single-core performance is only about half of that for the dRECT matrix, thus bandwidth saturation cannot be achieved. The cause of this failure is the generated assembly code: Although the compiler can vectorize the inner kernel along the matrix row, it accumulates the results into a single target register, which incurs the full `fmla` latency of 9 cy in every SIMD loop iteration. At $N_{nzr} = 27$, the inner loop has four iterations. Together with the latency of the required horizontal add instruction (`faddv`) of 49 cy, one row requires $4 \times 9 + 49$ cy $= 85$ cy to execute. Assuming again the maximum computational intensity, this translates into a maximum full-CMG bandwidth of

$$12 \text{ (cores)} \times 2.2 \text{ Gcy/s} \times 27 \times 12 \text{ byte}/85 \text{ cy} \approx 101 \text{ Gbyte/s} \tag{7}$$

and a maximum performance of 16.8 Gflop/s. Note that we did not consider the data transfers through the memory hierarchy since the overlapping part of the in-core execution dominates strongly. In practice, successive row executions can overlap slightly, which explains our measurement of 20 Gflop/s. Clearly the accumulation of partial sums into a single register is part of the problem. A solution to this problem will be discussed next.

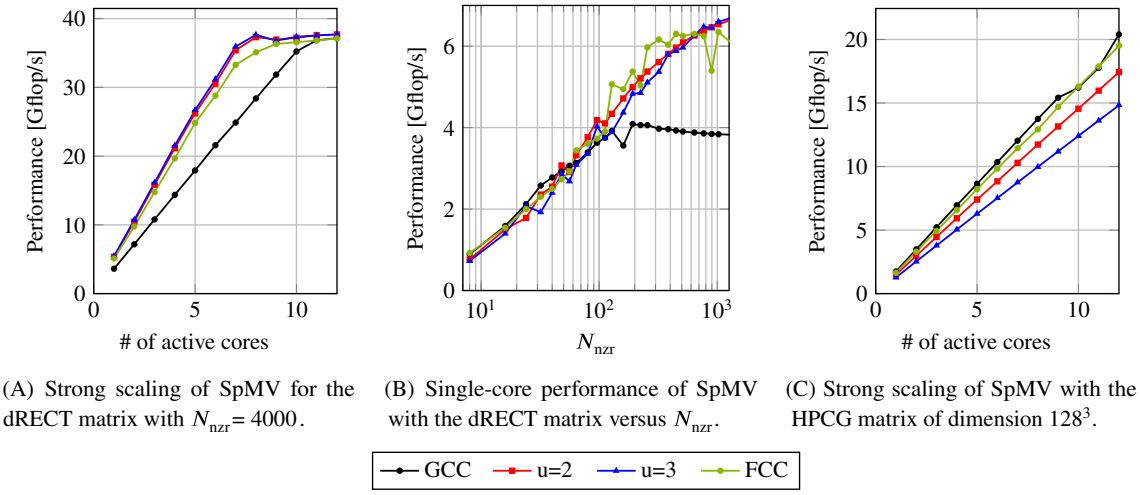
5.2 | Modulo variable expansion

MVE[14] accumulates partial sums into several registers, allowing for substantial overlapping of successive `fmla` instructions. The downside is that the computation of the final per-row result becomes more expensive since the reduction involves more registers. The FCC compiler can automatically employ MVE and produces two code paths, with and without MVE. Which path is taken is determined at runtime depending on the inner loop length. The GCC compiler does not employ MVE even when a `#pragma unroll` directive is used. Hence, from now on we revert to compiler intrinsics for all unrolled kernels to exert more control over the code generation.

We start by investigating the dRECT case. Figure 9(A) shows performance scaling at $N_{nzr} = 4000$. Unrolling by two or three with MVE clearly helps to boost the single-core performance and thus achieves stronger saturation[#] at around eight cores with GCC. As can be seen in Figure 9(B), this optimization is effective only if N_{nzr} is not too small, because the additional overhead for the final reduction cannot be amortized if the number of iterations in the inner loop is small. At an intensity of 1/6 flop/byte, a single-core performance of about 3 Gflop/s is required to saturate the CMG memory bandwidth with all cores. This becomes possible starting at $N_{nzr} \gtrsim 50$, but a significantly higher number is necessary to achieve strong saturation. Consequently, the CRS format is unable to yield best performance for matrices from many application fields: Figure 9(C) shows performance scaling with and without MVE for the HPCG matrix ($N_{nzr} = 27$). Saturation is not within reach.

The fundamental dilemma with the CRS format on A64FX is that SIMD vectorization and MVE must both be implemented within the inner loop. As a result, the inner loop becomes too short for effective in-core latency hiding on realistic matrices. Other storage formats such as SELL-C-σ can mitigate this problem.

[#]Strong saturation means saturation at a number of cores much smaller than the total number of cores.



(A) Strong scaling of SpMV for the dRECT matrix with $N_{nzr} = 4000$.

(B) Single-core performance of SpMV with the dRECT matrix versus N_{nzr}.

(C) Strong scaling of SpMV with the HPCG matrix of dimension 128^3.

FIGURE 9 Effect of MVE on the performance of SpMV using GCC with plain C code, explicit unrolling with GGC and MVE, and using the plain C code with the FCC compiler. MVE, modulo variable expansion; SpMV, sparse matrix-vector multiplication

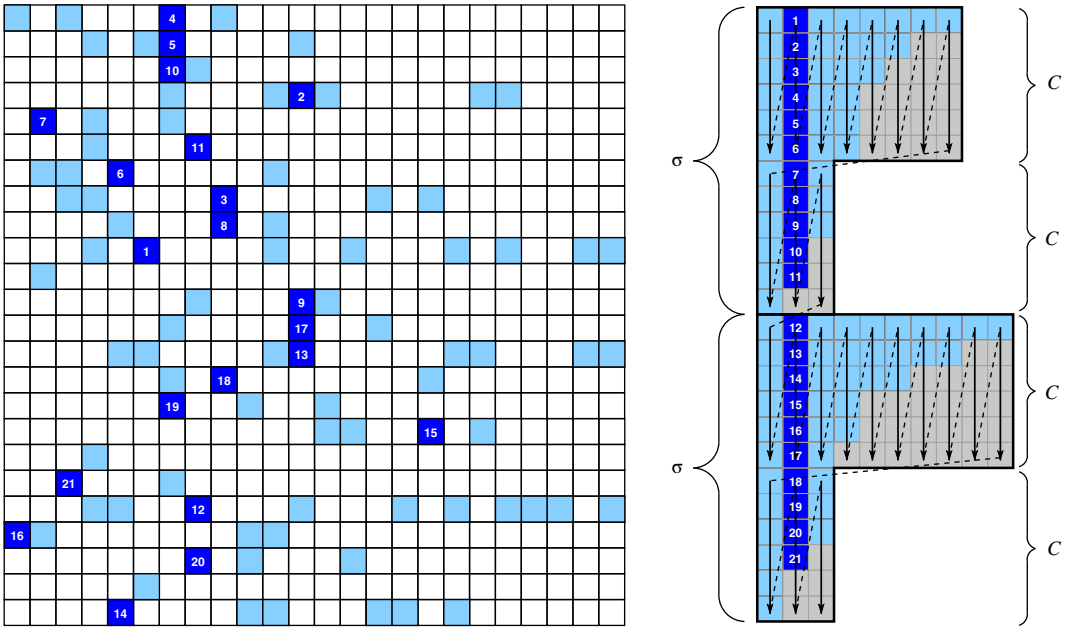
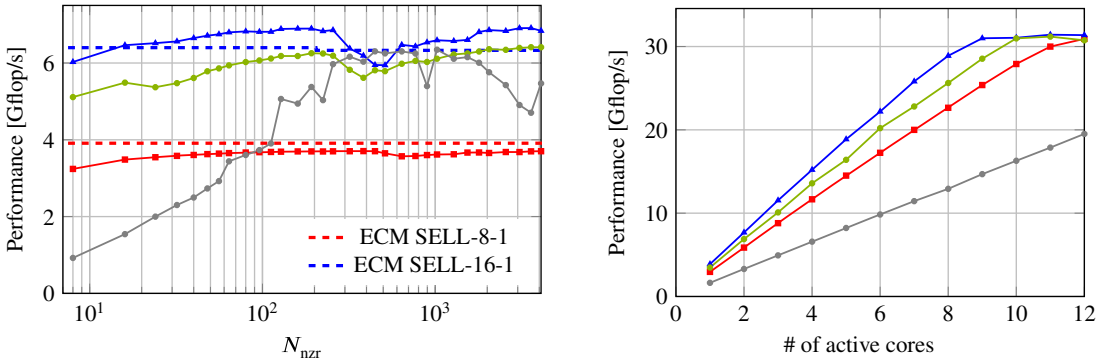


FIGURE 10 A sparse matrix with $N_r = 24$ rows (left) and the SELL-6-12 data structure generated from it (right). The blue boxes are nonzero entries, and the gray boxes are the zero fill-in. The arrows indicate the storage order. For illustration purposes, a column of nonzero entries is marked in dark blue in the SELL-6-12 figure; the corresponding entries are shown in the original matrix as well. Note that the permutation is applied to row and column indices alike

5.3 | SELL-C-σ

SELL-C-σ[13] is a sparse-matrix storage format suited for a broad range of architectures with wide SIMD or SIMT units. To convert a matrix to SELL-C-σ, its rows are first sorted within blocks of σ rows (the *sorting range*) according to the number of nonzero entries. Within each block of sorted rows, the nonzeros are stored in column-major format in chunks of height C (the *chunk size*). The columns of each chunk are zero-padded if necessary so that each row within a chunk has the same length. See Figure 10 for an illustration.

The inner loop of the corresponding SpMV code goes over one column of a chunk of height C (see Listing 3). This means that C should be a (small) multiple of the SIMD width, but large enough so that successive iterations of the loop, which accumulate into different target registers, can fill the bubbles in the `fmla` pipeline. Furthermore, no expensive horizontal reductions (`faddv`) are required. Due to the chunk padding, remainder loops



(A) Single-core SpMV performance with the dRECT matrix versus N_{nzr}, comparing the CRS format (gray, using FCC) with SELL-C-σ using different values of C and compilers.

(B) Strong scaling of SpMV within a CMG with the HPCG matrix.

FIGURE 11 SpMV performance with SELL-C-σ. SpMV, sparse matrix-vector multiplication

cannot occur and all SIMD lanes are filled. The second-innermost loop goes over the columns of a chunk and is as long as the chunk width, that is, N_{nzr} on average. Disadvantages of the SELL-C-σ format include possible excessive zero fill-in for very irregularly shaped matrices, and a potential impact of the row sorting on the access to the right-hand-side vector.

Figure 11(A) shows SpMV performance versus N_{nzr} using the dRECT matrix, comparing the CRS format (with the FCC compiler), SELL-8-1 with GCC, and SELL-16-1 with GCC and FCC. We also give the ECM-model predictions for the SELL cases. SELL-C-σ is able to keep close to the model even for small N_{nzr}, owing to the advantages shown above. Even with $C = 8$, which does not allow for mitigation of the pipeline stall on the `fmla` instruction, the performance loss at low N_{nzr} is small because of the absence of an expensive reduction after the loop across the chunk. At $C = 16$ the stall penalty is cut in half and leads to a significant performance boost. Note also that there is a distinctive drop in performance for the CRS format at $N_{nzr} \approx 4000$, which is caused by the right-hand-side vector not fitting in the L1 cache anymore. No such drop is visible for SELL-C-σ because the vector elements are reused along the columns of a chunk.

Figure 11(B) shows performance scaling of SpMV for the HPCG matrix, comparing the same setups as in Figure 11(A). As expected, the fastest single-core version (SELL-16-1) also exhibits the strongest saturation at nine cores. SELL-8-1 is also able to saturate but requires all cores on the CMG.

Finally, we compare the SELL-C-σ format with CRS on a range of matrices on the full A64FX chip (four CMGs) in Figure 12. Table 4 lists the properties of the test matrices. The matrix-specific memory-bound roofline limit, that is, assuming optimal reuse of the right-hand-side vector,[13] is shown for reference. We used three tuning parameters to find the best per-matrix performance: (i) Reverse Cuthill–McKee reordering, (ii) row-based versus nonzero-based load balancing, and (iii) σ in the range of 1–4096.

SELL-C-σ provides superior performance to CRS for almost all matrices. Exceptions exist where the access pattern to the right-hand-side vector changes for the worse compared with CRS. The difference between $C = 8$ and $C = 16$ is generally small. The significant gaps between measurement and model have a variety of reasons: Some matrices, such as scai1 and scai2, have a small N_{nzr} and a structure that leads to cache-unfriendly access patterns, thereby inhibiting saturation. In other cases, such as kkt-power, the matrix exhibits a strong imbalance of row lengths, making load balancing hard especially across CMGs (i.e., ccNUMA domains).

5.4 | SpMV and the sector cache

The sector-cache feature is expected to have a positive effect on the SpMV performance in cases where the cache is too small to ensure perfect reuse of the right-hand-side vector after it is loaded from memory. Restricting the number of cache ways used for the matrix data leaves more space for the vector, possibly increasing the computational intensity. Since invoking the sector cache comes with considerable overhead (see Section 3.3.3) we activate it outside the repetition loop. This is compatible with the structure of many sparse algorithms, where the same matrix is applied repeatedly to different vectors. Best results were obtained by allotting four ways of L2 and one way of L1 to the matrix data (nonzeros and index structures). The tuning space of Section 5.3 was enlarged by the chunk size ($C \in [1,128]$).

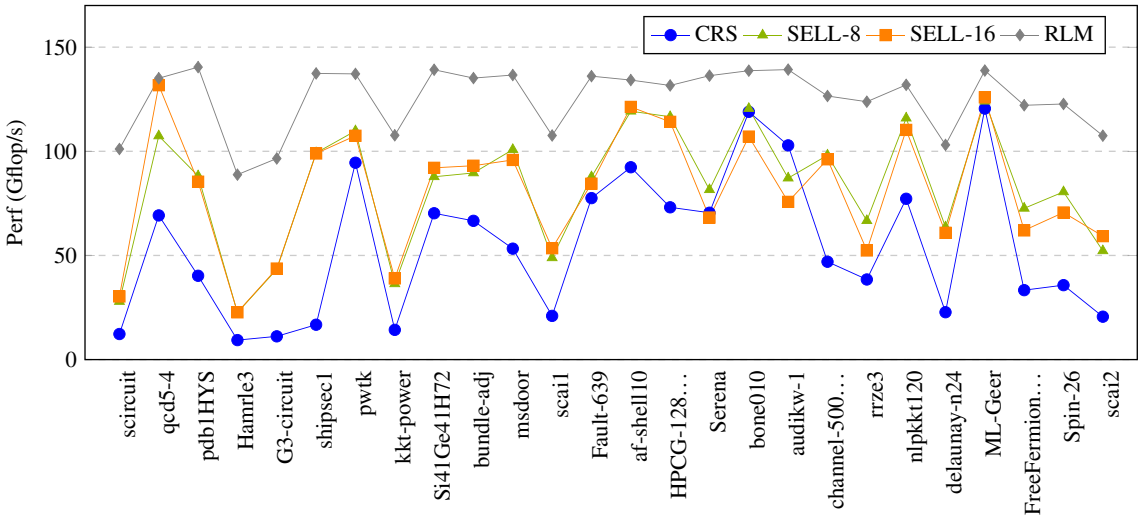


FIGURE 12 Influence of the sparse-matrix storage format on the SpMV performance for CRS, SELL-8-σ, and SELL-16-σ on the full A64FX chip (48 cores). The best value of σ was determined by exhaustive search in the range 1, … , 4096 for each matrix separately. Matrices are sorted from left to right in ascending order according to the number of nonzeros. The sector-cache feature was not used. Diamonds show the matrix-specific roofline limits (RLM). CRS, compressed row storage; SpMV, sparse matrix-vector multiplication

TABLE 4 Details of the benchmark matrices

Index	Matrix name	N_r	N_{nz}	N_{nzr}
1	scircuit	170,998	958,936	5.60788
2	qcd5_4	49,152	1,916,928	39.00000
3	pdb1HYS	36,417	4,344,765	119.30596
4	Hamrle3	1,447,360	5,514,242	3.80986
5	G3_circuit	1,585,478	7,660,826	4.83187
6	shipsec1	140,874	7,813,404	55.46378
7	pwtk	217,918	11,634,424	53.38900
8	kkt_power	2,063,494	14,612,663	7.08151
9	Si41Ge41H72	185,639	15,011,265	80.86266
10	bundle_adj	513,351	20,208,051	39.36497
11	msdoor	415,863	20,240,935	48.67212
12	scai1∗	3,405,035	24,027,759	7.05654
13	Fault_639	638,802	28,614,564	44.79411
14	af_shell10	1,508,065	52,672,325	34.92709
15	HPCG-128-128-128	2,097,152	55,742,968	26.58032
16	Serena	1,391,349	64,531,701	46.380672
17	bone010	986,703	71,666,325	72.63211
18	audikw_1	943,695	77,651,847	82.28490
19	channel-500x100x100-b050	4,802,000	85,362,744	17.776498
20	rrze3∗	6,201,600	92,527,872	14.92000
21	nlpkkt120	3,542,400	96,845,792	27.33903
22	delaunay_n24	16,777,216	100,663,202	5.999994
23	ML_Geer	1,504,002	110,879,972	73.72329
24	FreeFermionChain-26∗	10,400,600	140,616,112	13.52000
25	Spin-26∗	10,400,600	145,608,400	14.00000
26	scai2∗	22,786,800	160,222,796	7.03139

Note: N_r is the number of rows, N_{nz} is the number of nonzeros, and N_{nzr} is the average number of nonzeros per row. Most matrices were taken from the SuiteSparse Matrix Collection.[15] The matrices with ∗ come from other research projects.

In Figure 13, we show the impact of the sector cache on SpMV performance for the test matrices, comparing with the FCC compiler without sector cache and with GCC (which does not support sector cache). The largest benefit is observed with medium-sized matrices, where the additional cache space makes a difference for the right-hand-side-vector data reuse. Matrices like qcd5-4, which just about fit in the L2, suffer because the restricted cache space forces the matrix data into memory.

We include the GCC data in the plot because, although GCC does not support the sector cache on the A64FX, it produces better code from the intrinsics than FCC, which can be observed for some of the smaller matrices.

5.5 | Comparison with the Fujitsu SSL library

Fujitsu provides the "C-SSL II Thread-Parallel Capabilities" library, which contains SpMV routines for a variety of formats: compressed sparse columns (function `c_dm_vmvscc`), ELLPACK (function `c_dm_vmvse`), and Diagonal (DIA) storage. Since DIA is only suited for matrices with diagonal structures, we ignore it here. Figure 14 compares our SELL-C-σ and CRS implementations with the C-SSL library. The tuning space of Section 5.4 was enlarged by the sector-cache setting (on/off, same number of ways as in Section 5.4). The results show that the SpMV implementations in the current version of the C-SSL library are not competitive.

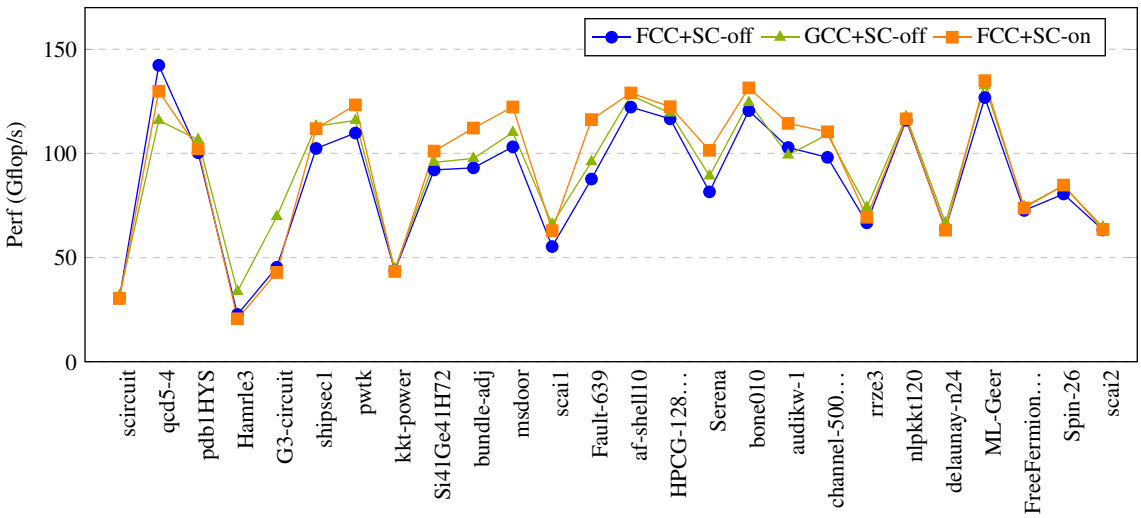


FIGURE 13 Influence of sector cache on the performance of sparse matrix-vector multiplication

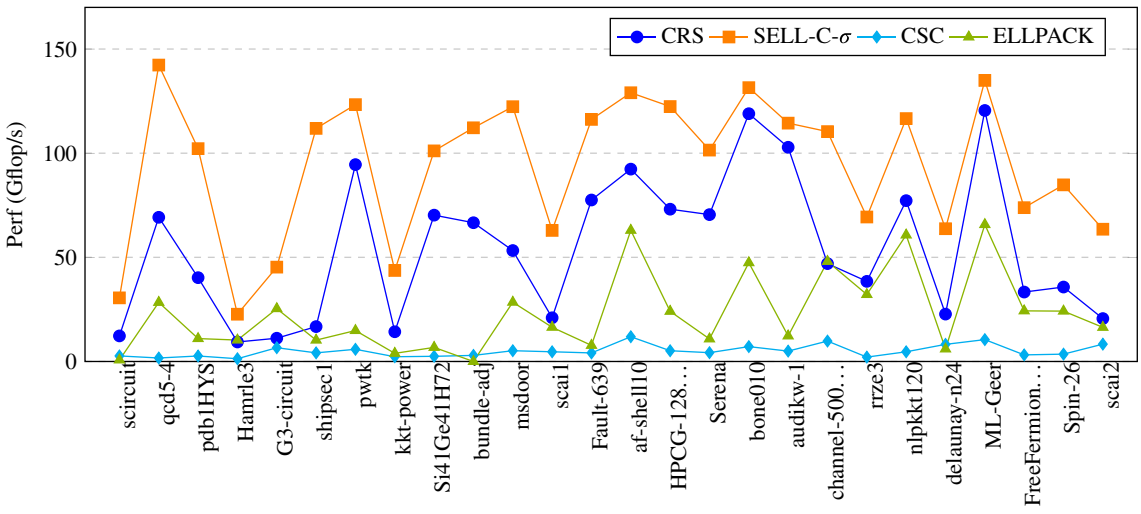


FIGURE 14 Performance comparison of SpMV with the CRS format, the SELL-C-σ format, and the Fujitsu SSL library using CSC and ELLPACK formats, respectively. CRS, compressed row storage; CSC, compressed sparse columns; SpMV, sparse matrix-vector multiplication

5.6 | Power consumption and tuning knobs

We explored two of the tuning knobs provided by the FX1000 system to optimize the energy consumption of the A64FX processor: the clock speed (2.0 GHz or 2.2 GHz) and the "eco" setting, which can be 0 (disabled), 1, or 2. For a strongly memory-bound code like the SELL-C-σ variant of SpMV on "benign" matrices, we expect that lowering the clock speed will have a negligible impact on the performance but at least a proportional influence on the power consumption (depending on the voltage scaling, for which details are undisclosed). Enabling eco mode, which includes disabling the FLB floating-point unit, should also be inconsequential for performance but probably advantageous for power.

Figure 15 shows the median, maximum and minimum performance and node energy consumption (in nJ/flop) for SpMV over all benchmark matrices, comparing all six combinations of the three eco settings and the two clock speeds. As expected, all settings have a minor impact on the performance. The low-clock-speed setting reduces the performance median by 2.7% but lowers the energy consumption by about 13%. The "eco=2" mode can additionally reduce the energy consumption by another 21%, for a total average of 31%. In light of the fact that not all SpMV executions are memory bandwidth bound, these are surprisingly large savings. Although the actual energy measurements fluctuate significantly because of the wide range of performance numbers, the general trend is the same even for the "hottest" and "coolest" cases.

Note that all measurements were taken on a single node of Fugaku. Significant statistical variations across nodes are expected, but a full coverage is beyond the scope of this article.

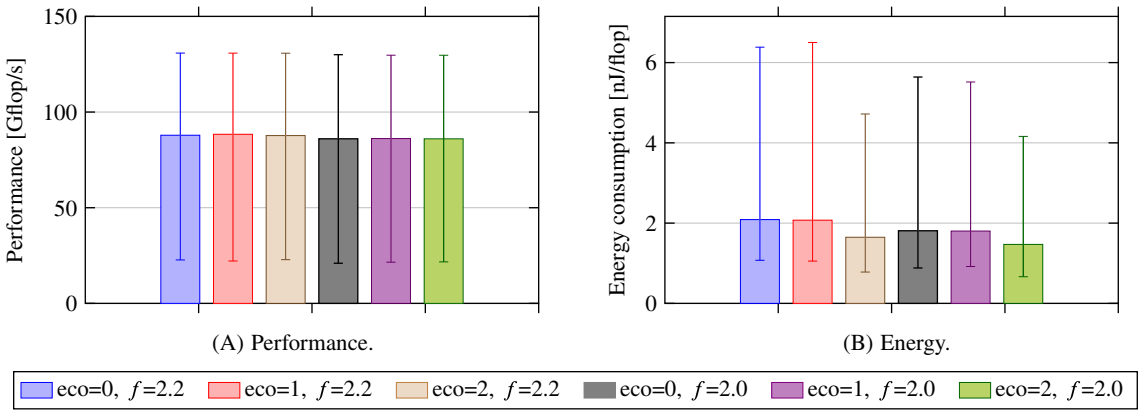


FIGURE 15 Comparison of the effects of different power settings on node performance and energy consumption for SpMV, reporting the median (bars), minimum and maximum (whiskers) over all benchmark matrices. The power dissipation varies between 130 and 190 W for the "hottest" setting (eco = 0 and f = 2.2 GHz) and between 76 and 133 W for the "coolest" setting (eco = 2, f = 2.0 GHz)

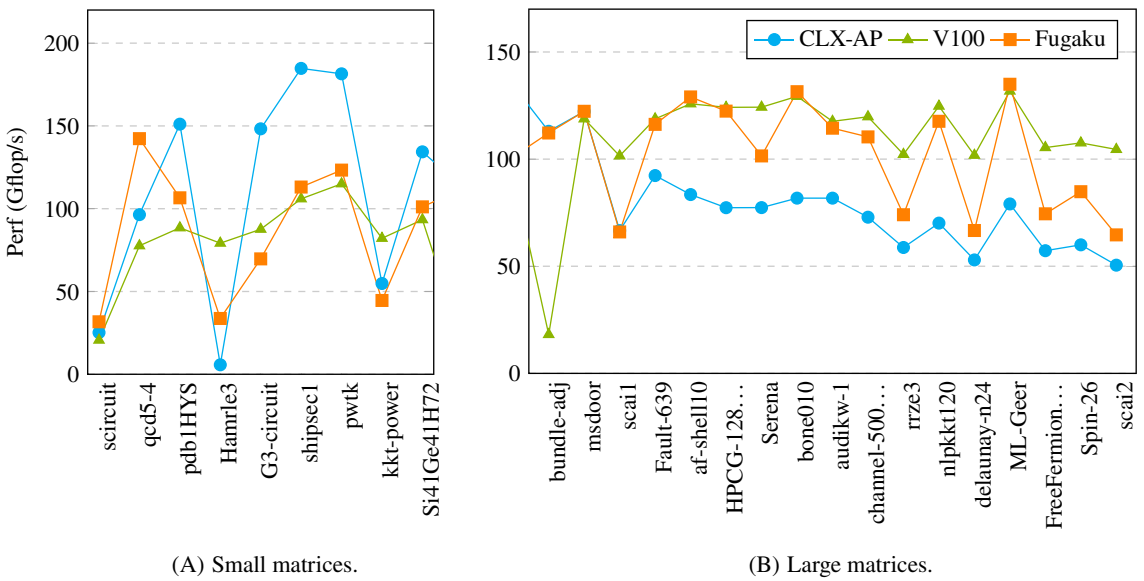
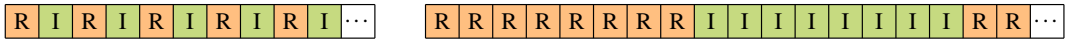


FIGURE 16 Comparison of SpMV performance on A64FX with other architectures. Note the different range of the vertical axis for the two graphs. SpMV, sparse matrix-vector multiplication



(A) RIRI. (B) RRII.

FIGURE 17 Illustration of RIRI and RRII data layouts for a 512-bit SIMD width. R's refer to real and I's to imaginary parts of double-precision complex numbers

5.7 | Comparison with other architectures

We choose one contemporary, high-end GPU and CPU architecture each to provide context for the SpMV performance of the A64FX: an NVIDIA V100 GPU and an Intel Cascade Lake AP (CLX-AP) node. On the V100 we use the GHOST library[16] for an efficient implementation of the SELL-C-σ format. The search space of Section 5.5 was extended on A64FX by the choice of compilers, GCC versus FCC.

Figure 16 shows the results on all three systems, separately for "small" and "large" matrices, the boundary being defined by the aggregate L2/L3 cache size of the CLX-AP. Unsurprisingly, the CLX-AP performs best for most small working sets. On in-memory matrices, Fugaku and the V100 show an approximate 1.6× – 2× speedup with respect to CLX-AP for "benign" matrices, which is in accordance with the memory-bandwidth ratio.[‖] With irregular matrices (e.g., Hamrle3, kkt_power, scai1/2, Spin-26, or FreeFermionChain-26), the GPU has a clear advantage due to its more effective latency hiding. The bundle-adj matrix has low performance on V100 due to GHOST only supporting row-based load balancing.

6 | CASE STUDY: LATTICE QCD DW KERNEL

6.1 | Introductory remarks

Understanding the strong interaction, one of the four known fundamental forces in nature, is one of the major challenges in physics. QCD is the quantum field theory of the strong interaction, which describes the interaction of quarks and gluons. Lattice QCD is a computer-friendly version of QCD, in which simulations are carried out on a regular lattice in Euclidean space-time. State-of-the-art research in Lattice QCD requires supercomputers such as Summit or Fugaku.

A significant part of the CPU time in Lattice QCD simulations is spent on solving a linear system of equations using iterative (multigrid) techniques. The key computational kernel is the application of the lattice Dirac operator to a quark-field vector Ψ. The quark field $\Psi(n)_{\alpha a}$ defined at lattice site n in a four-dimensional volume $V_4 = L_x \times L_y \times L_z \times L_t$ carries a spinor index $\alpha = 1, 2, 3, 4$ and a color index $a = 1, 2, 3$. The interaction is represented by SU(3) matrices $U_\mu(n)$, $n \in V_4$, carrying color indices. This matrices are defined on the links between adjacent sites n and $n + \hat{\mu}$, where $\hat{\mu}$ is the unit vector in direction μ.

Multiple formulations of quarks on the lattice exist. In this work, we focus on the DW fermion formulation,[17] in which the physical quark field $\Psi(n)$ lives on the four-dimensional boundary of a five-dimensional space-time lattice with volume $V_4 \times L_s$. The formulation involves a fermion field $\psi(n, s)_{\alpha a}$ that lives in five dimensions and carries an additional index $s = 1, \ldots, L_s$. The interaction $U_\mu(n)$ does not depend on the fifth dimension and is replicated along the s-direction. The performance-relevant part D of the DW Dirac operator acts on the fermion field as follows,[**]

$$\psi'(n,s)_{\alpha a} = (D\psi)(n,s)_{\alpha a} = \sum_{\mu=1}^{4}\sum_{\beta=1}^{4}\sum_{b=1}^{3}\{U_\mu(n)_{ab}(1+\gamma_\mu)_{\alpha\beta}\psi(n+\hat{\mu},s)_{\beta b} + U_\mu^\dagger(n-\hat{\mu})_{ab}(1-\gamma_\mu)_{\alpha\beta}\psi(n-\hat{\mu},s)_{\beta b}\}. \tag{8}$$

Here, the γ_μ are constant 4×4 Dirac matrices carrying spinor indices. The result of the projection $(1 \pm \gamma_\mu)\psi$ is a four-component spinor for each color. Up to multiplicative factors only two components each are independent. The number of input operands per site is 8×9 for the U-fields and 8×12 for the ψ-fields. The number of output operands is 1×12 for the ψ'-field per site. All these operands are complex numbers.

Grid[19] is a Lattice QCD software framework written in C++ with OpenMP and MPI parallelization. A variety of architectures are supported, including all Intel x86 SIMD extensions, Arm NEON and 512-bit SVE,[20,21] and GPGPUs. Grid achieves 100% SIMD efficiency on all architectures by combining template meta-programming and intrinsics where available. The data layout for complex numbers interleaves real and imaginary parts, that is, the numbers are stored as RIRI, where R/I stands for real/imaginary part (see Figure 17(A)). Using the SVE instruction set, hardware processing of $z_1 \pm z_2 \cdot z_3$ (complex multiply add) and $z_1 \pm iz_2$ is implemented using the `svcmla` and `svcadd` ACLE intrinsics, respectively. Key computational kernels such as the one emerging from Equation (8) have been specialized for the A64FX.[22]

[‖] Note that a Cascade Lake SP system has only half the memory channels of CLX-AP, so we expect the speedup to double in that case.

[**] The full operator is given in Reference 18, eqs. (2.5)–(2.7). Our D corresponds to their D^{II} with $M = 4$.

In this work, we study the performance of the DW kernel for different compilers. We also compare the interleaved data layout with an alternative "split" layout, where the real and imaginary parts are stored as RRII such that the R's and I's end up in separate vector registers. For example, for double precision and a 512-bit SIMD width, we have eight consecutive R's as shown in Figure 17(B). We use a subset of Grid,[23] which we extended for studying CLX-AP and the A64FX.[24] CLX-AP only supports real arithmetics, therefore the interleaved layout implies permutations. On the A64FX we use hardware support for the computation of the interleaved layout and real arithmetics otherwise.

6.2 | Code analysis

```
#define x_p 1 // x-plus  direction
#define x_m 2 // x-minus direction
#define y_p 3 // y-plus  direction
...
#pragma omp parallel for schedule(static)
for {t,z,y,x} = 1:{L_t-2,L_z-2,L_y-2,L_x-2}//collapsed loop over 4d space-time
{
  for(int s=0; s<L_s; ++s) //loop over fifth dimension
  {
    O[t][z][y][x][s] = R(x_p)·U[x_p][t][z][y][x]·P(x_p)·I[t][z][y][x+1][s] +
                       R(x_m)·U[x_m][t][z][y][x]·P(x_m)·I[t][z][y][x-1][s] +
                       R(y_p)·U[y_p][t][z][y][x]·P(y_p)·I[t][z][y+1][x][s] +
                       R(y_m)·U[y_m][t][z][y][x]·P(y_m)·I[t][z][y-1][x][s] +
                       R(z_p)·U[z_p][t][z][y][x]·P(z_p)·I[t][z+1][y][x][s] +
                       R(z_m)·U[z_m][t][z][y][x]·P(z_m)·I[t][z-1][y][x][s] +
                       R(t_p)·U[t_p][t][z][y][x]·P(t_p)·I[t+1][z][y][x][s] +
                       R(t_m)·U[t_m][t][z][y][x]·P(t_m)·I[t-1][z][y][x][s];
  }
}
```

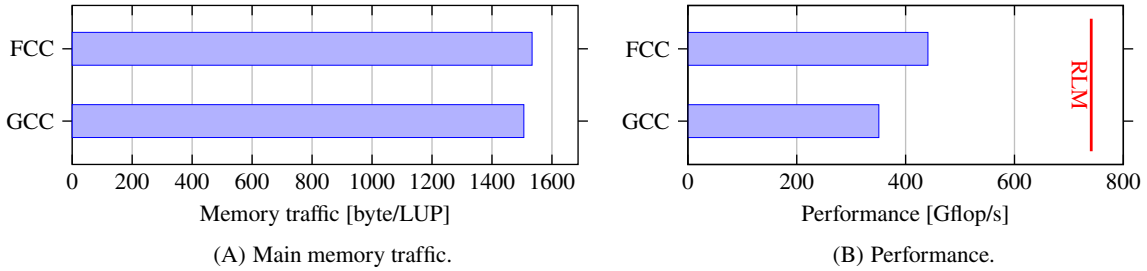
Listing 4: Simplified view of the domain wall kernel.

The DW kernel Equation (8) is a radius-1 star-shaped stencil[25] without the center element. The input and output of the stencil operation are the fermion fields $\psi(n,s)$ and $\psi'(n,s)$, respectively. The interaction matrices $U_\mu(n)$ and their inverses $U^\dagger_\mu(n)$ can be considered as variable stencil coefficients. Listing 4 shows a simplified version of the DW kernel omitting color and spinor indices as well as boundary conditions. The stencil code loops over the four dimensions x, y, z, and t, and the fifth dimension s. The input fermion $\psi(n,s)$ is stored in the array `I`, and the output fermion $\psi'(n,s)$ in the array `O`. The interaction matrices U and $U^\dagger$ are stored in `U` for the forward and backward directions in x, y, z, and t. Application of $(1 \pm \gamma_\mu)$ to ψ is arranged in two parts: spinor projection `P` and spinor reconstruction `R`.[††] The operations `P` and `R` are hard-coded and do not need any operands from memory. For each direction, the following computational sequence is applied:

1. `P` projects the (4×3)-component input fermion field in `I` to a (2×3)-component fermion field, where 2 means half-spinor and 3 is the number of colors.
2. Two matrix-vector multiplications are applied, one for each component of the half-spinor from step 1, using the same 3×3 matrix in `U`.
3. Reconstruction `R` of the (4×3)-component fermion field and addition to the output fermion field in `O` (if applicable), which are combined in one step.

The sum of all projections in step 1 contributes 96 flops. Each matrix-vector multiplication in step 2 requires $3 \cdot 3 = 9$ complex multiplications and $2 \cdot 3 = 6$ complex additions. Each of the complex multiplications is worth six flops, and a complex addition is worth two flops. Considering two matrix-vector multiplications in step 2 we have $2 \cdot (9 \cdot 6 + 6 \cdot 2) = 132$ flops per direction. Since there are eight directions we have a total of $8 \cdot 132 = 1056$ flops. Reconstruction and summation of intermediate results in step 3 adds another $7 \cdot 4 \cdot 3 \cdot 2 = 168$ flops. Thus, the theoretical total flop count is $96 + 1056 + 168 = 1320$. The actual flop count depends on the code implementation. However, all performance results reported in this study will be based on 1320 flops per lattice site update (LUP).

[††] See Reference 26 for the details of projection and reconstruction.



(A) Main memory traffic. (B) Performance.

FIGURE 18 (A) Main-memory traffic and (B) full-chip performance of the baseline implementation of the DW kernel compiled using `-Ofast` with GCC and FCC. The dimension of the lattice is 24 in each of the x, y, z, t directions and 8 in the s direction. The roofline model (RLM) performance estimate is shown in (B)

The flop count along with the data traffic to and from main memory can be used to construct a RLM performance limit. Figure 18(A) shows the main-memory data traffic of a baseline implementation of the DW kernel measured using the `likwid-perfctr` tool. This baseline implementation uses RIRI layout, ACLE intrinsics, and no prefetching. It can be seen that for the DW kernel we need approximately 1500 byte/LUP. The code intensity of the kernel can thus be estimated as $I = 1320/1500$ flop/byte $= 0.88$ flop/byte. According to the RLM, the performance estimate is given as $\min(p_{\text{peak}}, I \cdot b_{\text{Mem}})$, where p_{peak} is the peak flop rate and b_{Mem} is the saturated main-memory bandwidth of the hardware. For the A64FX, $p_{\text{peak}} = 3379.2$ Gflop/s and $b_{\text{Mem}} = 859$ Gbyte/s (see Table 1). The RLM thus predicts a memory-bound performance maximum of 756 Gflop/s.

Figure 18(B) shows the performance of the DW kernel on the full chip in comparison with the roofline prediction. The data structure in this version of the kernel uses the interleaved (RIRI) complex array layout.[‡‡] The code attains a performance close to 350 Gflop/s with GCC and 440 Gflop/s with FCC. The measurements fall short of the RLM limit by a factor of 2.1× and 1.7×, respectively. In the following sections, we will investigate the reasons for this significant deviation and explore optimization strategies.

6.3 | ECM analysis, layer conditions, and optimizations

The RLM predicts that the main-memory bandwidth is the performance bottleneck for the DW kernel, but we observe almost linear scaling up to 48 cores (not shown here), indicating bottlenecks at the single-core level. The single-core performance is 8.2 and 10.3 Gflop/s using GCC and FCC, respectively. We use the ECM performance model discussed in Section 4 to investigate this issue. To construct the model, we first need the ECM contributions $T_{\text{c\_OL}}$, $T_{\text{L1\_LD}}$, $T_{\text{L1\_ST}}$, T_{L2}, and T_{Mem} (see Section 4.1). These must be combined using the overlap hypothesis described in Section 4.2.

The L1-to-register contributions $T_{\text{L1\_LD}}$, $T_{\text{L1\_ST}}$ and the in-core computational contribution $T_{\text{c\_OL}}$ can be estimated by analyzing the assembly code with the OSACA tool. For the data delays in the memory hierarchy, that is, T_{L2} and T_{Mem}, we need the data volume V_i transferred over each data path i and the corresponding hardware bandwidth b_i (see Section 4) to plug into Equation (2). The data traffic can be modeled analytically, which can be done for stencil codes using LC.[11,27,28] Comparison of the prediction with performance counter measurements helps to validate the model and can reveal bottlenecks due to the code implementation and/or compiler code generation.

6.3.1 | Layer conditions

LC provide important information about which cache can hold which elements of the stencil and about the amount of data that must be transferred from and to a cache. The concept is based on reuse distance analysis, that is, the distance after which a certain element of the stencil is reused. Since stencils have a well-defined regular access pattern, this reuse distance can be determined analytically. The LC analysis assumes caches with infinite associativity and least recently used (LRU) replacement policy. Real caches, such as on the A64FX, have finite associativity and might only implement a pseudo-LRU policy. However, it has been shown in References 11,29 that for most stencil codes these assumptions do not hamper the quality of the predictions. For simplicity we assume in the following that the lattice is sufficiently large such that the working set does not fit into cache.

[‡‡] The SVE instruction set supports complex multiply add ($z_1 + z_2 \cdot z_3$), but lacks complex multiplication ($z_1 \cdot z_2$). Therefore, each matrix-vector multiplication requires three complex multiply add instructions for each row, including one multiplication adding zero ($0 + z_1 \cdot z_2$). The latter implies an additional $2 \cdot 3 \cdot 2 \cdot 8 = 96$ flops on top of the 1320 flops per LUP for the RIRI implementation.

TABLE 5 LC for the DW kernel for scalar and vectorized code

		$s_i >$	
Name	**V_j in byte/LUP**	**Scalar code**	**512-bit vectorized code**
no reuse	$(8 \cdot 9 + 8 \cdot 12 + w \cdot 12) \cdot 16$	0	0
LC_s	$(8 \cdot 9/L_s + 8 \cdot 12 + w \cdot 12) \cdot 16$	2880	11,520
LC_x	$(8 \cdot 9/L_s + 7 \cdot 12 + w \cdot 12) \cdot 16$	$2 \cdot L_s(8 \cdot 9/L_s + 8 \cdot 12 + 12) \cdot 16$	$d \cdot 8 \cdot L_s(8 \cdot 9/L_s + 8 \cdot 12 + 12) \cdot 16$
LC_y	$(8 \cdot 9/L_s + 5 \cdot 12 + w \cdot 12) \cdot 16$	$2 \cdot L_s \cdot L_x(8 \cdot 9/L_s + 7 \cdot 12 + 12) \cdot 16$	$d \cdot 8 \cdot L_s \cdot L_x(8 \cdot 9/L_s + 7 \cdot 12 + 12) \cdot 16$
LC_z	$(8 \cdot 9/L_s + 3 \cdot 12 + w \cdot 12) \cdot 16$	$2 \cdot L_s \cdot L_x \cdot L_y(8 \cdot 9/L_s + 5 \cdot 12 + 12) \cdot 16$	$8 \cdot L_s \cdot L_x \cdot L_y(8 \cdot 9/L_s + 5 \cdot 12 + 12) \cdot 16$
LC_t	$(8 \cdot 9/L_s + 1 \cdot 12 + w \cdot 12) \cdot 16$	$2 \cdot L_s \cdot L_x \cdot L_y \cdot L_z(8 \cdot 9/L_s + 3 \cdot 12 + 12) \cdot 16$	$4 \cdot L_s \cdot L_x \cdot L_y \cdot L_z(8 \cdot 9/L_s + 3 \cdot 12 + 12) \cdot 16$

Note: To determine the data traffic from a certain cache i, its size s_i (in bytes) has to be compared with each condition starting from the bottom row (LC_t) to top row (no reuse). The first condition that is met by the cache i determines the layer condition it satisfies, and the next higher hierarchy level j has a data transfer volume of V_j. In our case the write-allocate factor is $w = 2$. The d factor accounts for the data layout and will be discussed in Section 6.3.4. For the RIRI (RRII) layout we have $d = 1$ ($d = 2$).
Abbreviations: DW, domain wall; LC, layer conditions.

Data traffic analysis

To construct the LC we need to analyze the data access patterns in the stencil code (Listing 4). We can neglect projections `P` and reconstructions `R` since these do not contribute to data traffic. The innermost loop is along the s dimension, followed by the x, y, z, and t dimensions. The elements of the array `U` are 3×3 matrices, whose entries are of type `double complex` (16 bytes), while the elements of the arrays `I` and `O` are 4×3 `double complex` matrices. Within a LUP and without data reuse we need to load eight `U` elements of dimension 3×3 each and eight `I` elements of dimension 4×3 each, and then store one `O` element of dimension 4×3. In total, we touch $(8 \cdot 9 + 8 \cdot 12 + 12) \cdot 16$ byte = 2.88 kB in each LUP.

The elements of `U[*][t][z][y][x]`[§§] are independent of the s loop and are touched again after a single LUP. Therefore, if a cache i of size s_i can hold all the elements required to compute a single lattice site, that is, if $s_i > 2.88$ kB, then these elements can be reused in the cache i. In this case, the next higher[¶¶] memory hierarchy level j has to deliver eight `U` elements only once per traversal of the s loop, and therefore the data traffic V_j will correspond to $(8 \cdot 9/L_s + 8 \cdot 12 + w \cdot 12) \cdot 16$ byte/LUP. Here, w is the write-allocate factor: We have $w = 2$ if write allocation applies, which is the case in our implementations because there is neither a zero fill intrinsic nor a compiler built-in function. The condition of optimal reuse in the s dimension is labeled LC_s in the following.

Once all the elements in the innermost s loop are traversed, the next loop is along the $x$dimension. Reuse of the element `I[t][z][y][x-1][s]` can happen in this loop, since it was touched two x iterations ago. In order to satisfy this reuse condition, a cache has to hold all the elements touched in the s-loop iteration for two iterations of the x loop. Therefore, the LC_x condition reads $s_i > 2 \cdot L_s \cdot (8 \cdot 9/L_s + 8 \cdot 12 + 12) \cdot 16$ bytes. If this condition is satisfied by cache i then memory level j only has to transfer seven elements of the array `I` instead of eight, translating to $V_j = (8 \cdot 9/L_s + 7 \cdot 12 + w \cdot 12) \cdot 16$ byte/LUP.

The other conditions LC_y, LC_z, and LC_t are constructed along the same lines. A summary of LC along each dimension is shown in Table 5 (see the scalar code column).

Serial code and vectorization

Stencil codes are typically vectorized along the innermost dimension. However, in the Grid Lattice QCD framework,[19] vectorization is implemented along the 4d space-time dimensions for two reasons: First, the extent L_s of the fifth dimension would have to be a multiple of the VL (VL = 4 for the RIRI layout in double precision), which imposes restrictions on the choice of L_s that are specific to the underlying hardware architecture. Second, other Lattice QCD kernels do not use a fifth dimension and use the vectorization scheme in 4d space-time dimensions.

An MPI-style partitioning scheme is applied to define the mapping of lattice sites to SIMD registers. One 512-bit SIMD register of the A64FX can hold four complex numbers in double precision; therefore, the lattice is divided into four partitions. This is realized by cutting the outermost two dimensions in the 4d space-time in half, that is, each of the four local (virtual) partitions has a size of $L_x \times L_y \times L_z/2 \times L_t/2 \times L_s$. Figure 19 illustrates the partitioning of the lattice and the mapping of lattice sites to SIMD registers. Note that the site numbering is not lexicographic: Adjacent elements of a SIMD register belong to different partitions. Therefore, a change in site numbering implies a change in the data access pattern, which must be taken into account in the construction of the LC. Access to all four partitions is identical, and we can account for this by only considering

[§§] Here $*$ refers to all eight directions.

[¶¶] Higher means farther away from the core, for example, L2 is higher than L1.

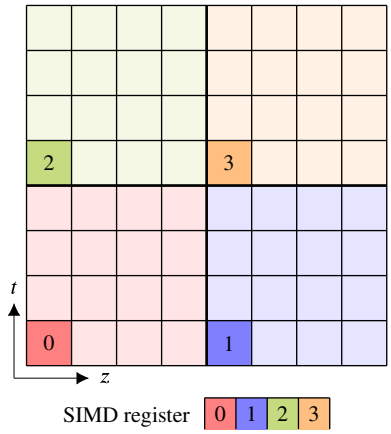


FIGURE 19 Illustration of the vectorization scheme for a lattice of size 8 each in the outermost dimensions *t* and *z*. Complex elements from four different partitions are packed into one SIMD register. The ordering of lattice sites is shown with numbers

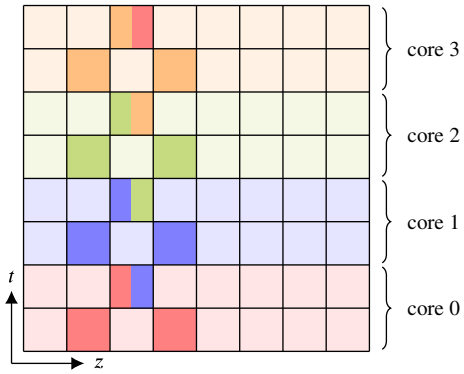


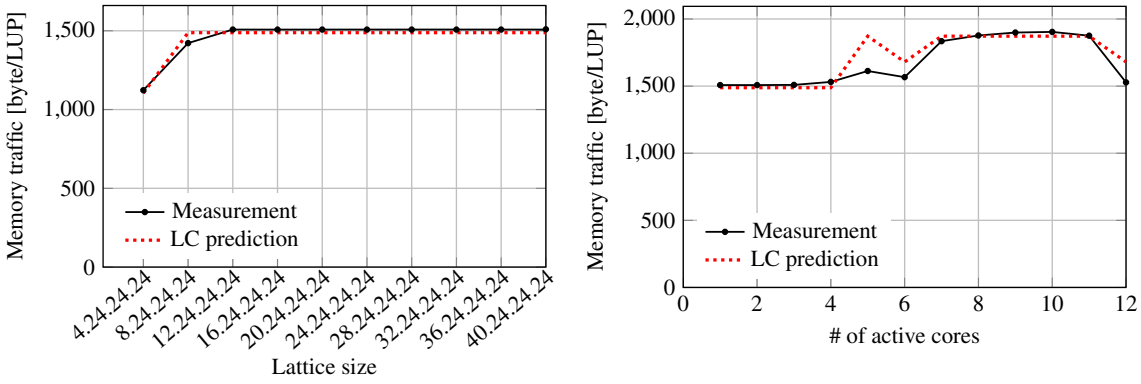
FIGURE 20 Illustration of data sharing between four cores attached to the same shared cache for a local (virtual) lattice size of 8 each in the outermost dimensions *t* and *z* with periodic boundary conditions. Each color represents a different core. The dark-colored elements show the stencil accesses at a specific time. We observe that two cores access the same element in the *t* direction

the access pattern within one local partition and multiplying the number of elements touched at each access by a factor of four. For instance, the LC_t condition in Table 5 becomes $s_i > \mathbf{4} \cdot 2 \cdot L_x \cdot L_y \cdot (L_z/\mathbf{2}) \cdot L_s \cdot (8 \cdot 9/L_s + 3 \cdot 12 + 12) \cdot 16$ bytes. The highlighted factors 4 and 2 reflect the changes due to vectorization. LC for scalar and vectorized code are shown in Table 5. It can be seen that this kind of vectorization makes the LC more stringent. However, compared with the traditional inner-loop vectorization approach it does not imply redundant L1-to-register loads and/or shuffle operations.

Figure 21(A) shows the LC effect by changing the lattice size L_x on a single core, keeping the extent of the other dimensions constant. For $L_x = 4$ the L2 cache satisfies LC_z. As L_x increases, the L2 cache violates the LC_z condition (see the linear dependence between LC_z and L_x in Table 5).

Multicore layer conditions

The basic LC principle applied to serial code also holds for multicore. However, care should be taken when the cache *i* under consideration is a shared cache. In this case, the per-core cache size s_i and possible sharing of data between cores has to be taken into account. The first aspect is easily integrated into LC by dividing the size S_i of the shared cache by the number of active cores *n*, that is, $s_i = S_i/n$. Proper handling of data sharing among cores requires knowledge of the loop-scheduling technique and the lattice dimensions. For the DW kernel, thread parallelization is done via default OpenMP static scheduling along the outermost loop over the collapsed 4d space-time dimensions (see Listing 4). Thus in most cases there is no data sharing among the cores. However, the data traffic can be reduced by sharing of elements of the stencil array `I`. This is the case, for example, when the lattice dimensions are chosen to be a small multiple of the number of cores that share the same cache. Figure 20 illustrates data sharing between cores. In the outermost direction *t* two cores share a stencil element. These sharing effects have to be taken into account when adapting the LC to the multicore environment.



(A) Memory traffic for different lattice sizes.

(B) Memory traffic for different number of cores.

FIGURE 21 (A) Influence of lattice size on memory traffic on a single core with $L_s = 8$ and (B) influence of number of cores on memory traffic for lattice size $24^4 \times 8$. Predictions are shown as red dashed lines

Figure 21(B) shows the influence of the number of cores on the LC due to the shared L2 cache. The lattice size is $24^4 \times 8$, that is, a local (virtual) partition has a size of $24 \times 24 \times 12 \times 12$ in the 4d space-time dimensions. Using only a single core, the L2 satisfies LC_y. As the number of cores increases, the available cache per core decreases. At five cores, LC_y is violated for L2, and only LC_x is satisfied. This explains the increase in the traffic between main memory and L2. At six cores, the traffic decreases due to sharing of the data between cores as the local outermost dimension (12) is a small multiple of six and a condition similar to Figure 20 applies. For seven to eleven cores there is no more sharing and the traffic stays at the level corresponding to LC_x. At twelve cores it decreases again due to sharing. Note that the model (shown with dotted lines) suffers some loss in accuracy especially at the boundaries of LC transitions. This is due to the assumptions made in the model, that is, infinite cache associativity with LRU and perfectly synchronized "lockstep" execution across cores.

6.3.2 | Initial optimizations

Before diving into ECM performance predictions we first look into initial optimizations based on the LC and the insights gained from the microarchitecture analysis (see Section 3.1).

Prefetching

Figure 22(A,B) compare the measured L2 and main memory traffic of the vectorized serial code with the LC predictions (shown as dashed lines). For the baseline implementation the memory traffic is in line with the prediction. However, the measured L2 traffic exceeds the prediction by about 2×. A closer inspection reveals that the increase in traffic is due to the hardware prefetchers not moving the correct elements into the cache. The DW kernel has a complex access pattern, which repeats only after all the elements in the innermost loop were accessed, that is, after the computation of one site. This requires access to 2.88 kB of data for scalar code and about 11.5 kB for vectorized code. It is extremely difficult if not impossible for the prefetchers to correctly predict the next elements.

The deficiencies of the hardware prefetching mechanics can be overcome by software prefetching. This decreases the L2 traffic, which is now within 30% of the prediction (see Figure 22(A)). The single-core performance improves by a factor of 1.3× for both GCC and FCC as seen in Figure 22(C). For the following discussions we assume that software prefetching is applied.

Instruction order

In Section 3.1, we showed that the OoO window of the A64FX is small and suffers from inefficiencies in instruction reordering. Therefore, the compiler has to support the hardware in hiding instruction latencies by proper instruction ordering. For the DW kernel implementation guidance is given to the compiler at the level of the source code (by explicit reuse of variables and ordering of operations). However, we noticed that GCC rearranges the intended order of instructions in a manner that deteriorates performance and also introduces unnecessary register spills. In order to mitigate these undesired effects we apply optimization flag `-O1` instead of `-Ofast`. This keeps the intended order of instructions intact and also minimizes spilling. The performance improves by 1.3× as can be seen in Figure 22(C). FCC `-O1` and `-Ofast` arrange the instructions as intended and performance is comparable to GCC `-O1`.

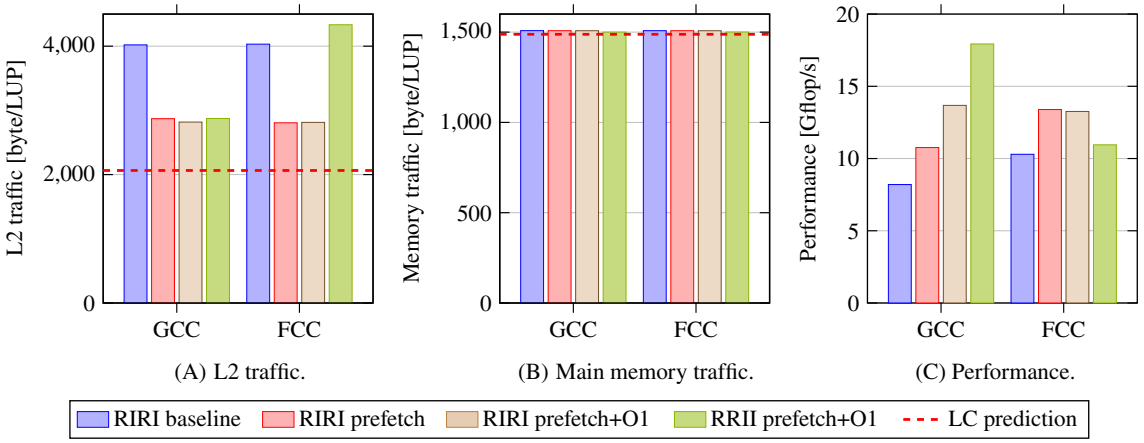


FIGURE 22 Influence of software prefetching and compiler code generation on the domain wall kernel. Data traffic and predictions (red dashed lines) as well as single-core performance are shown for a lattice size of $24^4 \times 8$. The RIRI prefetch implementation applies software prefetching on top of the RIRI baseline implementation. Codes were compiled with `-Ofast`. The RIRI prefetch+O1 and RRII prefetch+O1 implementations were compiled with `-O1` each

TABLE 6 ECM contributions in cy/LUP to the DW kernel implementations for lattice size $24^4 \times 8$

	GCC						FCC					
Implementation	T_{c_OL}	T_{L1_LD}	T_{L1_ST}	T_{L2}	T_{Mem}	T_{ECM}	T_{c_OL}	T_{L1_LD}	T_{L1_ST}	T_{L2}	T_{Mem}	T_{ECM}
`RIRI-prefetch`	168.0	25.6	3	35.3	15.9	168.0	168.0	33	3	35.3	15.9	168.0
`RRII-prefetch`	70.8	34.4	20.4	35.3	15.9	70.8	85.5	45.2	37.3	35.3	15.9	85.5

Note: We used the `-O1` compiler flag.
Abbreviations: DW, domain wall; ECM, Execution-Cache-Memory.

6.3.3 | ECM-model prediction for interleaved RIRI layout

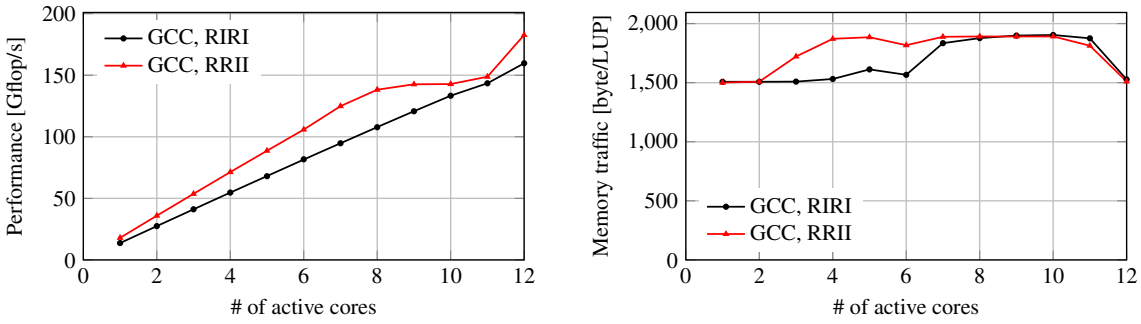
Using the LC we can estimate the data traffic V_i and derive T_i using Equation (2). Along with the in-core contributions T_{c_OL}, T_{L1_LD}, and T_{L1_ST} obtained using the OSACA tool, we can assemble the contributions using the overlap hypothesis (see Section 4.2) to finally determine the expected performance.

For a lattice size of $24^4 \times 8$ we can see from Table 5 that the A64FX satisfies LC_s in the L1 cache, which means $V_{L2} = 2064$ byte/LUP, out of which 1872 byte/LUP is due to loading data from L2 to L1 and 192 byte/LUP is due to storing data from L1 to L2. The L2 load throughput is 64 byte/cy and the store throughput is 32 byte/cy (see Table 1), which results in $T_{L2} = (1872/64 + 192/32)$ cy/LUP $= 35.25$ cy/LUP. The T_{Mem} contribution can be derived in a similar fashion. All contributions are summarized in Table 6. They can be combined with the overlap hypothesis (D) in Figure 6 to finally arrive at the ECM prediction T_{ECM}. Irrespective of the compiler we find

$$T_{ECM} = \max(168, f(25.6, 3, 35.3, 15.9)) \text{ cy/LUP} = 168 \text{ cy/LUP}. \quad (9)$$

The T_{ECM} runtime corresponds to a single-core performance of 17.3 Gflop/s. However, we see from Figure 22(C) that the measured single-core performance falls short of the prediction by 20% and only attains about 13.6 Gflop/s. The deviation from the ECM model can be expected on the A64FX. The model assumes that the OoO execution overlaps multiple loop iterations and hides all latencies. However, as the DW kernel takes almost 170 cy per LUP it is not possible for the OoO logic to sufficiently overlap the iterations (see Section 3.1). On a single CMG we would thus attain a performance of $12 \cdot 13.6 = 163.2$ Gflop/s, which is the measured performance shown in Figure 23(A). The corresponding throughput attained by the code on a single CMG is $163.2 \cdot V_{Mem}/1320 = 184$ Gbyte/s, which does not saturate the CMG memory bandwidth of 227 Gbyte/s (see Table 1).

In order to achieve saturation we have to improve the single-core performance by at least 15%, which warrants a closer look at the current bottleneck. As can be seen in Table 6, the bottleneck of the RIRI implementation is the in-core overlapping part T_{c_OL}. OSACA reveals that this is predominantly due to high occupation of floating-point ports on the A64FX caused by the high cost of SVE instructions for complex arithmetics (`fcmla` and `fcadd`). These instructions block the floating-point ports for three and two cycles, respectively. Furthermore, the `fcmla` instruction



(A) Strong scaling performance. (B) Memory traffic.

FIGURE 23 Scaling performance and main-memory traffic of the RIRI and RRII implementations as a function of the number of active cores (GCC option `-O1`). The lattice size is $24^4 \times 8$

is imbalanced between the FLA and FLB ports; two out of the three cycles are scheduled to the FLA port. OSACA indicates that FLB has a 35% lower occupancy than FLA. One option to mitigate the pipeline imbalance is to avoid the SVE complex instructions and to use ordinary floating-point instructions instead. In this case, the cost would only be two cycles for a complex FMA and one cycle for a complex ADD operation. We discuss the implementation of this option in the following subsection.

6.3.4 | Split RRII layout

Balanced pipeline usage is achieved by a change of the data layout. Instead of using the interleaved RIRI layout, we switch to the split RRII layout. In the case of 512-bit wide registers and in double precision we store eight real parts in memory, followed by eight imaginary parts (see Figure 17(B)). Vectorized code operates on eight sites in parallel instead of four sites as in the RIRI layout (see Figure 19). This has to be factored into the LC analysis. The necessary modifications to the LC are shown in Table 5, where we now have $d = 2$ ($d = 1$ for the RIRI layout). This leads to tighter conditions and LC breaking earlier when scaling up the number of cores. This is shown in Figure 23(B), where LC_γ breaks earlier with RRII than with RIRI.

The implementation of the RIRI layout guides the compiler to efficiently use the 32 floating-point vector registers without spilling. However, for the RRII layout the number of registers is insufficient and spilling of intermediate results occurs. This is reflected in the L1-to-register contributions $T_{\mathrm{L1\_LD}}$ and $T_{\mathrm{L1\_ST}}$ shown in Table 6. GCC minimizes register spills when using `-O1`. FCC produces almost 2× more spills than GCC, resulting in lower performance (see Figure 22(C)). We therefore exclude FCC from further analysis.

The RRII implementation still outperforms RIRI despite the register spills. This is because the overlapping in-core time $T_{\mathrm{c\_OL}}$, which is the bottleneck of the RIRI layout, reduces by more than a factor of two (see GCC in Table 6). The lower cost of ordinary (noncomplex) floating-point instructions and the balanced pipeline usage are the main reasons for this reduction. Still, the single-core performance falls short of the model prediction by at least a factor of 2. We speculate that this gap is caused partly by in-core inefficiencies rooted in the dependencies between loads and stores [4, sec. 7.5], which are not taken into account in the model. This can be corroborated by the fact that for GCC with fewer spills the model deviates by a factor of 2.3×, while for FCC the deviation is 3.1×. Beyond the dependencies between loads and stores, the inefficient OoO execution also contributes to this deviation. To test the actual limit of the in-core execution for the code we constructed a benchmark with the same instruction dependency chain as the GCC code and measured the runtime while keeping all the data in the L1 cache. This yielded 113 cy/LUP, which is 1.6× higher than the in-core prediction seen in Table 6.

Using GCC, the RRII implementation already saturates the main-memory bandwidth using eight cores (see Figure 23(A)). The sudden increase in performance when using 12 cores is due to sharing of the data in L2 among cores (discussed in Section 6.3.1). It correlates with a drop in main-memory traffic as seen in Figure 23(B). We also observe a decrease in main-memory traffic for the RIRI implementation, but it is not accompanied by an increase in performance because this version cannot saturate the memory bandwidth.

On one CMG the RRII implementation achieves 182 Gflop/s, which corresponds to a memory bandwidth of almost 205 Gbyte/s. On the full chip (four CMGs) we attain 712 Gflop/s, which is a 12% improvement over RIRI.

6.4 | Energy consumption and tuning knobs

Figure 24 shows the performance and energy consumption applying the power knobs described in Section 5.6 to the DW kernel. For the highest performance setting (eco = 0 and $f = 2.2$ GHz) we see that the energy consumption of the RIRI implementation is almost 20% higher than for RRII.

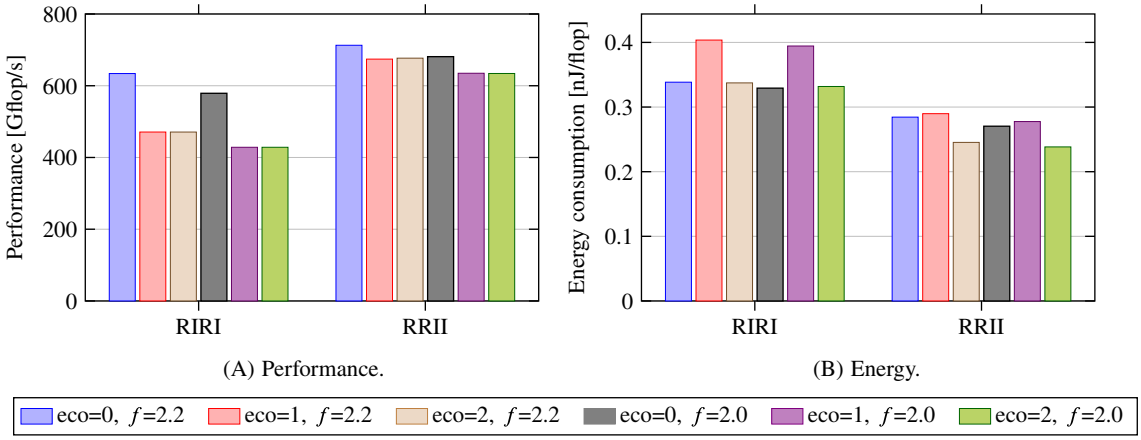


FIGURE 24 Comparison of performance and energy consumption of domain wall kernel implementations for different power and frequency settings on a full chip of Fugaku. The lattice size is $24^4 \times 8$. The energy consumption of the RIRI implementation is almost 20% higher than RRII. The energy consumption of RRII can be reduced without significant loss in performance by lowering the clock frequency and switching on eco mode

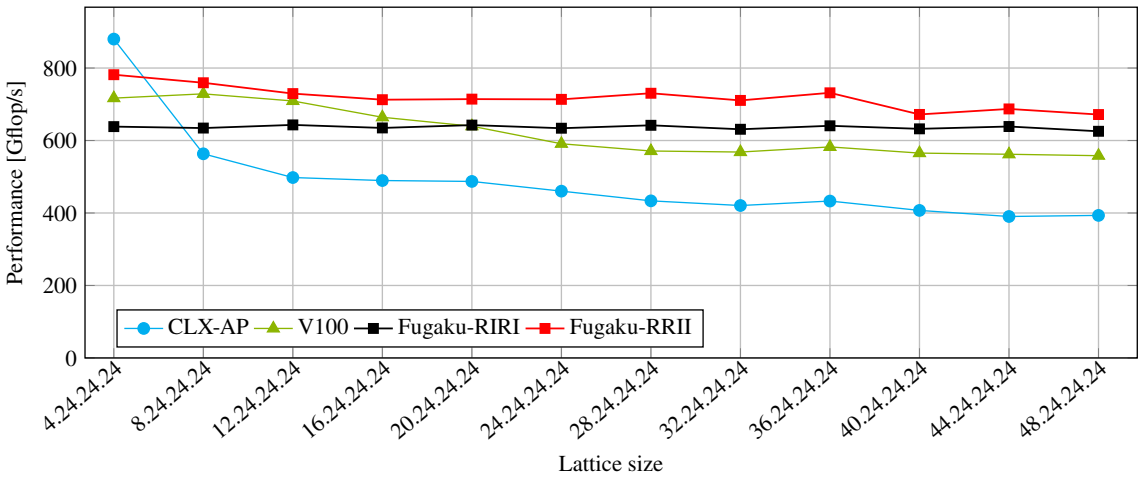


FIGURE 25 Performance comparison of the domain wall kernel on Fugaku, Intel Cascade Lake AP, and NVIDIA V100 for various lattice sizes with $L_s = 8$

Furthermore, the energy consumption of RRII can be reduced further without significant loss in performance by lowering the clock frequency and switching on eco mode. However, for RIRI the performance drops drastically when eco mode is activated since here the bottleneck is the FLA pipeline (see Section 6.3.3). Eco mode turns off the FLB pipeline, which increases the load on FLA and hence the runtime. Regardless of the implementation, the power consumption of the DW kernel is about 205 W at the highest power and frequency settings. It reduces to about 145 W with eco = 2 and $f = 2.0$ GHz.

6.5 | Comparison with other architectures

In this section, we briefly compare the DW kernel performance among the A64FX, an Intel Cascade Lake CPU (CLX-AP), and a V100 GPU. Figure 25 shows this comparison for various lattice sizes. The size of the lattice is varied only along the innermost dimension x. The experiments on the V100 were conducted using the Grid Lattice QCD framework. We used our own implementations[24] on the other architectures. For CLX-AP and V100 we used the RIRI layout. This layout performs best on CLX-AP. For the V100 this is the only layout currently available in Grid. The results are qualitatively similar to that of the SpMV performance comparison shown in Section 5.7 as both kernels are bandwidth bound. For smaller lattice sizes the CLX-AP has the advantage of large caches, while for larger lattice sizes both the A64FX and V100 have a 1.5× performance advantage due to the higher memory bandwidth. The A64FX has a slight performance advantage over the V100, however, we did not do a detailed inspection and analysis of the V100 code to see if there are any optimization opportunities.

6.6 | Further optimization options

The LC analysis suggests further optimization opportunities. For example, cache blocking minimizes LC violations and facilitates data reuse in the caches. Thread scheduling techniques increase data sharing between cores. Zero fill instructions could be used to reduce the data traffic by avoiding write-allocate transfers. Another potential improvement is to first cut the inner dimensions (s and x) for vectorization rather than the current approach of cutting outermost dimensions. This will further relax the LC. However, these optimizations involve architecture-specific tuning parameters and code paths and thus impact code maintainability.

We have not yet investigated the use of single precision, which is often sufficient depending on the algorithm in which the kernel is embedded. This will be the subject of future work.

7 | CONCLUSIONS

7.1 | Summary and outlook

We have further improved the ECM machine model for the A64FX CPU introduced in Reference 1 and showed its applicability to the Fugaku processor. We validated the model with simple streaming kernels and could observe a high accuracy for in-memory datasets. The memory hierarchy is partially overlapping, allowing for a substantial single-core memory bandwidth with optimized code. Long floating-point instruction latencies and limited OoO execution capabilities were identified as the main culprits of poor performance and lack of bandwidth saturation. Vector intrinsics and manual unrolling are often required to achieve high performance.

Furthermore, we applied the ECM model to SpMV to identify the impact of A64FX proprietary performance features such as the sector cache and power-related tuning knobs. The SELL-C-σ matrix storage was shown to achieve performance and memory-bandwidth saturation superior to the standard CRS format. A comparison with state-of-the-art CPU and GPU architectures showed that for large, memory-bound SpMV datasets, the A64FX can outperform Intel's CLX-AP by a factor of two and is on par with NVIDIA's V100.

Finally, we used the ECM model for a comprehensive analysis of the Lattice QCD DW kernel in a subset of the Grid framework. We showed that both compilers used for this work (FCC and GCC) exhibit a lack of quality in code optimization. A change of the data layout can achieve a better balance of the port pressure and thus increase the performance significantly. When comparing the energy consumption of data layouts, the split RRII data layout proved to be almost 20% more energy efficient without a noticeable loss in performance. The A64FX shows a speedup of 1.5× over the CLX-AP for problem sizes exceeding both architectures' last-level cache and comparable performance to a V100.

The present work opens up many interesting opportunities for future research. For example, load-balancing issues together with the ccNUMA characteristics of the A64FX warrant further investigation. The insight gained from the SpMV analysis can be applied to applications such as Chebyshev filter diagonalization or linear solvers. Further optimizations to the QCD kernels have already been discussed in Section 6.6. Last but not least a more detailed analysis of the mechanisms for power tuning is of great interest to all applications.

7.2 | Related work

Since the A64FX CPU is a very recent design, the amount of performance-centric research is limited. Dongarra[30] reported on basic architectural features, HPC benchmarks (HPL, HPCG, HPL-AI) and the software environment of the Fugaku system. Poenaru and McIntosh-Smith[31] presented results on the effect of using wide vector registers and compared the performance and cache behavior of the A64FX for HPC benchmarks to the ThunderX2 platform. Both Odajima et al.[32] and Jackson et al.[33] investigated benchmarks, full applications and proxy apps in comparison to Intel and other Arm-based systems but did not use performance models for analysis. Gupta et al.[34] and Brank et al.[35] investigated stencil codes, proxy applications, SpMV and memory-bound fluid solvers on several Arm-based platforms including A64FX but did not provide detailed and validated performance models. Heybrock et al.[36] and Joó et al.[26] analyzed data traffic properties and data layout options for QCD kernels on the Intel Xeon Phi architecture. Kodama et al.[37] studied a comprehensive set of power-tuning knobs for STREAM and DGEMM kernels on Fugaku.

ACKNOWLEDGMENTS

We thank Daniel Richtmann for providing us with the V100 benchmarks of the DW kernel and Julian Hammer for useful discussions regarding cache modeling. This work used computational resources of the supercomputer Fugaku provided by RIKEN through the HPCI System Research Project (Project ID: hp200261). We are indebted to HLRN for providing access to their Lise cluster. This work was supported in part by KONWIHR, by DFG in the framework of SFB/TRR 55 and by MEXT as "Program for Promoting Researches on the Supercomputer Fugaku" (Simulation for basic science: from fundamental laws of particles to creation of nuclei).

DISCLAIMER

The results obtained on the evaluation environment in the trial phase do not guarantee the performance, power, and other attributes of the supercomputer Fugaku at the start of its public use operation.

DATA AVAILABILITY STATEMENT

The data that support the findings of this study are available from the corresponding author upon reasonable request.

ORCID

Christie Alappat https://orcid.org/0000-0003-4548-8727
Georg Hager https://orcid.org/0000-0002-8723-2781

REFERENCES

1. Alappat C, Laukemann J, Gruber T, et al. Performance Modeling of Streaming Kernels and Sparse Matrix-Vector Multiplication on A64FX. Paper presented at: Proceedings of the 2020 IEEE/ACM Performance Modeling, Benchmarking and Simulation of High Performance Computer Systems (PMBS), Atlanta, GA, USA; November 12, 2020:1-7; IEEE. https://doi.org/10.1109/PMBS51919.2020.00006
2. ARM C Language Extensions for SVE. https://developer.arm.com/documentation/100987/0000/. Accessed September 28, 2020.
3. Hofmann J. ibench – Measure Instruction Latency and Throughput; 2018. https://github.com/RRZE-HPC/ibench.
4. Fujitsu Limited A64FX Microarchitecture Manual 1.3. Fujitsu Limited; 2020. https://github.com/fujitsu/A64FX/blob/1b3071af0369ee02b1752b7556e949050349985d/doc/A64FX_Microarchitecture_Manual_en_1.3.pdf.
5. Treibig J, Hager G, Wellein G. LIKWID: A Lightweight Performance-Oriented Tool Suite for x86 Multicore Environments. Paper presented at: Proceedings of the 2010 39th International Conference on Parallel Processing Workshops, San Diego, GA, USA; September 13, 2010:207-216; IEEE. https://doi.org/10.1109/ICPPW.2010.38
6. Gruber T, Eitzinger J, Hager G, Wellein G. RRZE-HPC/likwid: likwid-5.1.0; 2020. https://github.com/RRZE-HPC/likwid/.
7. Grant RE, Levenhagen M, Olivier SL, DeBonis D, Pedretti KT, Laros JH III. Standardizing Power Monitoring and Control at Exascale. *Computer*. 2016;49(10):38-46. https://doi.org/10.1109/MC.2016.308
8. Laukemann J, Hammer J, Hofmann J, Hager G, Wellein G. Automated Instruction Stream Throughput Prediction for Intel and AMD Microarchitectures. Paper presented at: Proceedings of the 2018 IEEE/ACM Performance Modeling, Benchmarking and Simulation Of High Performance Computer Systems (PMBS); 2018:121-131. https://doi.org/10.1109/PMBS.2018.8641578.
9. Laukemann J, Hammer J, Hager G, Wellein G. Automatic Throughput and Critical Path Analysis of x86 and ARM Assembly Kernels. Paper presented at: Proceedings of the 2018 IEEE/ACM Performance Modeling, Benchmarking and Simulation of High Performance Computer Systems (PMBS), Denver, CO, USA; November 12, 2018:121-131; IEEE. https://doi.org/10.1109/PMBS49563.2019.00006
10. Hofmann J, Alappat C, Hager G, Fey D, Wellein G. Bridging the Architecture Gap: Abstracting Performance-Relevant Properties of Modern Server Processors. *Supercomput Front Innov*. 2020;7(2):54–78. https://doi.org/10.14529/jsfi200204
11. Stengel H, Treibig J, Hager G, Wellein G. Quantifying Performance Bottlenecks of Stencil Computations using the Execution-Cache-Memory model. Paper presented at: Proceedings of the 29th ACM on International Conference on Supercomputing ICS '15; 2015; ACM, New York, NY. https://doi.org/10.1145/2751205.2751240
12. Gropp WD, Kaushik DK, Keyes DE, Smith BF. Towards Realistic Performance Bounds for Implicit CFD Codes. In: Keyes D, Periaux J, Ecer A, Satofuka N, Fox P, eds. *Parallel Computational Fluid Dynamics*. Vol 2000. North Holland: Elsevier; 1999:241-248. https://wgropp.cs.illinois.edu/bib/papers/pdata/1999/pcfd99/gkks.ps
13. Kreutzer M, Hager G, Wellein G, Fehske H, Bishop AR. A Unified Sparse Matrix Data Format for Efficient General Sparse Matrix-Vector Multiplication on Modern Processors with Wide SIMD Units. *SIAM J Sci Comput*. 2014;36(5):C401-C423. https://doi.org/10.1137/130930352
14. Lam M. Software Pipelining: An Effective Scheduling Technique for VLIW Machines. Paper presented at: Proceedings of the ACM SIGPLAN 1988 conference on Programming Language design and Implementation PLDI '88; 1988:318-328; ACM, New York, NY. https://doi.org/10.1145/53990.54022
15. Davis TA, Hu Y. The University of Florida Sparse Matrix Collection. *ACM Trans Math Softw*. 2011;38(1):1:1-1:25. https://doi.org/10.1145/2049662.2049663
16. Kreutzer M, Thies J, Röhrig-Zöllner M, et al. GHOST: Building Blocks for High Performance Sparse Linear Algebra on Heterogeneous Systems. *Int J Parallel Program*. 2017;45:1046-1072. https://doi.org/10.1007/s10766-016-0464-z
17. Kaplan DB. A method for simulating chiral fermions on the lattice. *Phys Lett B*. 1992;288(3):342-347. https://doi.org/10.1016/0370-2693(92)91112-M
18. Furman V, Shamir Y. Axial symmetries in lattice QCD with Kaplan fermions. *Nucl Phys B*. 1995;439:54-78. https://doi.org/10.1016/0550-3213(95)00031-M
19. Boyle P, Yamaguchi A, Cossu G, Portelli A. Grid: a next generation data parallel C++ QCD library. *PoS (LATTICE 2015)*; Trieste, Italy: SISSA; 2016:023. https://doi.org/10.22323/1.251.0023
20. Georg P, Meyer N, Pleiter D, Solbrig S, Wettig T. SVE-enabling lattice QCD Codes. Paper presented at: Proceedings of the 2018 IEEE International Conference on Cluster Computing (CLUSTER), Belfast, UK; 2018:623-628. https://doi.org/10.1109/CLUSTER.2018.00079
21. Meyer N, Pleiter D, Solbrig S, Wettig T. Lattice QCD on upcoming ARM architectures. *PoS (LATTICE 2018)*; Trieste, Italy: SISSA; 2019:316. https://doi.org/10.22323/1.334.0316
22. Georg P, Meyer N, Pleiter D, Solbrig S, Wettig T. Lattice QCD on QPACE 4; 2020. https://conference-indico.kek.jp/event/113/contributions/2139/attachmen%ts/1391/1545/aplat2020_lqcd_on_qpace4_meyer_v2.pdf.
23. Boyle P, Yamaguchi A. GridBench – Single CPU benchmarks cutting down Grid; 2020. https://github.com/paboyle/GridBench.
24. Meyer N, Alappat C. GridBench – AVX512 and A64FX extensions; 2021. https://github.com/nmeyer-ur/GridBench/tree/intrinsics.
25. Hornich J, Hammer J, Hager G, Gruber T, Wellein G. Collecting and Presenting Reproducible Intranode Stencil Performance: INSPECT. *Supercomput Front Innov*. 2019;6(3):4–25. https://doi.org/10.14529/jsfi190301

26. Joó B, Smelyanskiy M, Kalamkar DD, Vaidyanathan K. Chapter 9 - Wilson Dslash Kernel from Lattice QCD Optimization. In: Reinders J, Jeffers J, eds. *High Performance Parallelism Pearls Volume Two: Multicore and Many-core Programming Approaches*. Vol 2. Boston, MA: Morgan Kaufmann; 2015:139-170. https://doi.org/10.1016/B978-0-12-803819-2.00023-9
27. Rivera G, Tseng CW. Tiling Optimizations for 3D Scientific Computations. Paper presented at: ACM/IEEE Conference on Supercomputing, IEEE Computer Society (SC '00), Dallas, TX, USA; 2000:32; IEEE. https://doi.org/10.1109/sc.2000.10015
28. Hammer J, Eitzinger J, Hager G, Wellein G. Kerncraft: A Tool for Analytic Performance Modeling of Loop Kernels. In: Niethammer C, Gracia J, Hilbrich T, Knüpfer A, Resch MM, Nagel WE, eds. *Tools for High Performance Computing 2016*. Cham, Swizerland: Springer International Publishing; 2017:1-22. https://doi.org/10.1007/978-3-319-56702-0_1
29. Alappat CL, Seiferth J, Hager G, Korch M, Rauber T, Wellein G. YaskSite: Stencil Optimization Techniques Applied to Explicit ODE Methods on Modern Architectures. Paper presented at: Proceedings of the 2021 IEEE/ACM International Symposium on Code Generation and Optimization (CGO), Seoul, South Korea; 2021:174-186. https://doi.org/10.1109/CGO51591.2021.9370316
30. Dongarra J. *Report on the Fujitsu Fugaku System. Technical Report ICL-UT-20-06*, Knoxville, TN: University of Tennessee, Department of Electrical Engineering and Computer Science, Innovative Computing Laboratory; University of Tennessee, Knoxville, Oak Ridge National Laboratory: 2020. https://www.icl.utk.edu/files/publications/2020/icl-utk-1379-2020.pdf.
31. Poenaru A, McIntosh-Smith S. The Effects of Wide Vector Operations on Processor Caches. Paper presented at: Proceedings of the 2020 IEEE International Conference on Cluster Computing (CLUSTER), Kobe, Japan; 2020:531-539. https://doi.org/10.1109/CLUSTER49012.2020.00076
32. Odajima T, Kodama Y, Tsuji M, Matsuda M, Maruyama Y, Sato M. Preliminary Performance Evaluation of the Fujitsu A64FX Using HPC Applications. Paper presented at: Proceedings of the 2020 IEEE International Conference on Cluster Computing, Kobe, Japan; 2020:523-530. https://doi.org/10.1109/CLUSTER49012.2020.00075
33. Jackson A, Weiland M, Brown N, Turner A, Parsons M. Investigating Applications on the A64FX. Paper presented at: Proceedings of the 2020 IEEE International Conference on Cluster Computing (CLUSTER), Kobe, Japan; September 14; 2020:549-558. https://doi.org/10.1109/CLUSTER49012.2020.00078
34. Gupta N, Ashiwal R, Brank B, Peddoju SK, Pleiter D. Performance Evaluation of ParalleX Execution model on Arm-based Platforms. Paper presented at: Proceedings of the 2020 IEEE International Conference on Cluster Computing (CLUSTER), Kobe, Japan; 2020:567-575. https://doi.org/10.1109/CLUSTER49012.2020.00080
35. Brank B, Nassyr S, Pouyan F, Pleiter D. Porting Applications to Arm-based Processors. Paper presented at: Proceedings of the 2020 IEEE International Conference on Cluster Computing (CLUSTER), Kobe, Japan; 2020:559-566. https://doi.org/10.1109/CLUSTER49012.2020.00079
36. Heybrock S, Joó B, Kalamkar DD, et al. Lattice QCD with Domain Decomposition on Intel® Xeon Phi Co-Processors. *SC '14: Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*. New York City, NY, USA: IEEE Press; 2014:69-80. https://doi.org/10.1109/SC.2014.11
37. Kodama Y, Odajima T, Arima E, Sato M. Evaluation of Power Management Control on the Supercomputer Fugaku. Paper presented at: Proceedings of the 2020 IEEE International Conference on Cluster Computing (CLUSTER), Kobe, Japan; 2020:484-493. https://doi.org/10.1109/CLUSTER49012.2020.00069

How to cite this article: Alappat C, Meyer N, Laukemann J, et al. Execution-Cache-Memory modeling and performance tuning of sparse matrix-vector multiplication and Lattice quantum chromodynamics on A64FX. *Concurrency Computat Pract Exper*. 2022;34(20):e6512. doi: 10.1002/cpe.6512